DNN

Contents

1 Module 1: Introduction to Deep Learning 2

2 Module 2: Artificial Neural Networks & Perceptrons 11

3 Module 3: Linear Neural Networks for Regression 18

4 Module 4: Linear Neural Networks for Classification 24

5 Module 5: Deep Feed-Forward Neural Networks (DFNN) 30

6 Module 6: Convolutional Neural Networks (CNN) 36

7 Module 7: Recurrent Neural Networks (RNN) 42

8 Module 8: Attention Mechanism 48

9 Module 9: Transformer 53

10 Module 10: Optimizers 58

11 Module 11: Regularisation 63

1 Module 1: Introduction to Deep Learning

Question 1

Question 1: Data Storage Calculation for Image Datasets
A deep learning researcher is preparing a dataset of high-resolution color images for training a Convolutional Neural Network (CNN). The dataset consists of $50,000$ images. Each image has a resolution of 1024×1024 pixels and uses the RGB color model (3 channels). The pixel values are stored as 8-bit integers (1 byte per channel per pixel).

1. Calculate the size of a single image in Megabytes (MB).
2. Calculate the total size of the dataset in Gigabytes (GB).
3. If the data is converted to floating-point numbers (32-bit float, 4 bytes per value) for training, what will be the new total dataset size in GB?

Solution

Concept: Images are represented as 3D arrays (Height $\times$ Width $\times$ Channels). The total storage size is the product of total pixels, channels, and the size of the data type used to store each value.
Step-by-Step Solution:
Part 1: Size of a single image (8-bit integer)

1. **Identify Dimensions:** Height $H = 1024$, Width $W = 1024$, Channels $C = 3$.
2. **Calculate Total Pixels per Channel:** $1024 \times 1024 = 1,048,576$ pixels.
3. **Calculate Total Bytes:** Total Values $= H \times W \times C = 1,048,576 \times 3 = 3,145,728$ values. Since each value is 8-bit (1 byte): Total Bytes $= 3,145,728$ bytes.
4. **Convert to Megabytes (MB):** 1 MB $=$ 1024 $\times$ 1024 bytes $=$ $1,048,576$ bytes.
$$\text{Size} = \frac{3,145,728}{1,048,576} = 3 \text{ MB}.$$

Part 2: Total Dataset Size (8-bit)

1. **Total Images:** $N = 50,000$.
2. **Total Size in MB:** $50,000 \times 3$ MB $= 150,000$ MB.
3. **Convert to Gigabytes (GB):** 1 GB $= 1024$ MB.
$$\text{Total Size} = \frac{150,000}{1024} \approx 146.48 \text{ GB}.$$

Part 3: Total Dataset Size (32-bit float)

1. **Scaling Factor:** A 32-bit float uses 4 bytes, whereas an 8-bit integer uses 1 byte. Thus, the size increases by a factor of 4.

2. **Calculate New Size:**

$$\text{New Size} = 146.48 \text{ GB} \times 4 = 585.92 \text{ GB}.$$

Final Answer: 1. 3 MB per image. 2. $\approx$ 146.48 GB for the dataset (8-bit). 3. $\approx$ 585.92 GB for the dataset (32-bit float).

Question 2

Question 2: Flattening and Feature Vectors
A simple feed-forward neural network is being designed to classify grayscale images of handwritten digits. The input images have dimensions 28×28 pixels.

1. If the image is "flattened" into a 1D feature vector, what is the dimensionality (length) of this vector?

2. If the input layer connects to a hidden layer with 128 neurons, how many weights (excluding biases) will be in this connection matrix?

3. If we add bias terms for the hidden layer, what is the total number of trainable parameters for this layer?

Solution

Concept: Neural networks typically require input data to be a 1D vector (for dense layers). Flattening converts a 2D matrix into a 1D vector. The number of parameters in a dense layer is (Input Size$\times$Output Size)$+$Output Size (for biases).
Step-by-Step Solution:
Part 1: Feature Vector Dimension

1. **Image Dimensions:** 28×28.

2. **Flattening:**

$$d = 28 \times 28 = 784.$$

The feature vector has a length of 784.

Part 2: Number of Weights

1. **Input Size (n_{in}):** 784.

2. **Hidden Layer Size (n_{out}):** 128.

3. **Weight Matrix Calculation:**

$$W = n_{in} \times n_{out} = 784 \times 128 = 100,352.$$

Part 3: Total Parameters (including Bias)

1. **Bias Vector Size:** Equal to the number of neurons in the hidden layer (128).
2. **Total Parameters:**

$$P = W + b = 100,352 + 128 = 100,480.$$

Final Answer: 1. 784 dimensions. 2. 100,352 weights. 3. 100,480 total parameters.

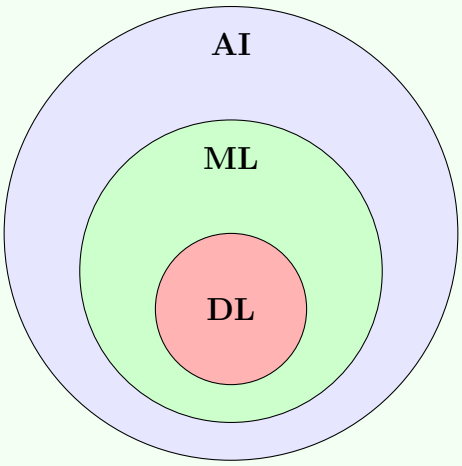
Question 3

Question 3: Hierarchical Logic Sets (Venn Diagram Concept)
Consider the three fields: Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).

1. Mathematically express the relationship between these sets using set notation (subset $\subset$).
2. A system uses a fixed set of "If-Then" rules (Expert System) to diagnose a disease. Which of the sets (AI, ML, DL) does this system belong to?
3. A system learns to recognize cats by manually extracted features (e.g., edge histograms) fed into a logistic regression model. Does this belong to DL? Explain.

Solution

Concept: The relationship is hierarchical. AI is the broadest field. ML is a subset of AI that learns from data. DL is a subset of ML that uses multi-layered neural networks.



Step-by-Step Solution:
Part 1: Set Notation Since Deep Learning is a type of Machine Learning, and Machine Learning is a type of AI:

$$\text{DL} \subset \text{ML} \subset \text{AI}$$

Part 2: Expert System Classification An "If-Then" expert system performs a task requiring intelligence (diagnosis) but does **not** learn from data (the rules are hard-coded). Therefore, it belongs to **AI**, but **not ML** or **DL**.

$$\text{Set: AI} \setminus \text{ML}$$

Part 3: Manual Feature Extraction This system learns from data (Logistic Regression), so it is **ML**. However, it relies on *manual* feature extraction rather than automated feature learning via multiple layers of abstraction. Therefore, it is **Traditional ML**, not **DL**.
Final Answer: 1. DL $\subset$ ML $\subset$ AI. 2. Belongs to AI only. 3. No, it is ML but not DL because it uses manual feature extraction instead of hierarchical feature learning.

Question 4

Question 4: Video Data Bandwidth Calculation
An autonomous vehicle records video from 4 cameras simultaneously. Each camera records at 1920×1080 resolution (Full HD), 30 frames per second (fps), with 3 color channels (RGB). The pixel depth is 8 bits (1 byte) per channel.

1. Calculate the uncompressed data rate generated by **one** camera in Megabytes per second (MB/s).
2. Calculate the total data rate for all 4 cameras in MB/s.
3. If the vehicle has a 1 TB SSD dedicated to video storage, how many minutes of uncompressed footage can it store?

Solution

Concept: Video data rate = Frame Size $\times$ Frame Rate. Frame Size = Width $\times$ Height $\times$ Channels $\times$ Byte Depth.
Step-by-Step Solution:
Part 1: Single Camera Data Rate

1. **Frame Size:** $1920 \times 1080 \times 3$ bytes $= 6,220,800$ bytes.
2. **Data Rate per Second:** $6,220,800$ bytes/frame $\times$ 30 fps $=$ $186,624,000$ bytes/s.
3. **Convert to MB/s:** $186,624,000/(1024 \times 1024) \approx 177.98$ MB/s.

Part 2: Total Data Rate (4 Cameras)

1. 177.98 MB/s $\times 4 \approx 711.92$ MB/s.

Part 3: Storage Duration

1. **Total Storage:** 1 TB $\approx 1,000,000$ MB (using decimal definition for simplicity, or 1024^2 MB $= 1,048,576$ MB). Let's use binary standard (1024^2). Total Capacity $= 1,048,576$ MB.

2. **Total Seconds:**

$$T_{sec} = \frac{1,048,576 \text{ MB}}{711.92 \text{ MB/s}} \approx 1472.88 \text{ seconds.}$$

3. **Total Minutes:**

$$T_{min} = \frac{1472.88}{60} \approx 24.55 \text{ minutes.}$$

Final Answer: 1. 177.98 MB/s. 2. 711.92 MB/s. 3. Approx 24.5 minutes.

Question 5

Question 5: Matrix Multiplication Dimensions
In a Deep Learning model, we have the following matrix operations:

$$Z = W \cdot X + b$$

Where:

- X is the input batch of shape (N, d_{in}).
- W is the weight matrix.
- Z is the output of shape (N, d_{out}).
- b is the bias vector.

Given $N = 32$ (Batch size), $d_{in} = 100$, and $d_{out} = 50$:

1. Determine the required shape of the weight matrix W. (Assume standard linear algebra convention $Z = XW^T$ or similar, clarify the orientation based on dimensions).
2. Determine the shape of the bias vector b for broadcasting.
3. How many scalar multiplications are performed to compute Z (ignoring bias addition)?

Solution

Concept: Matrix multiplication compatibility requires inner dimensions to match. If $Z = X \cdot W$, then $(N, d_{out}) = (N, d_{in}) \times (d_{in}, d_{out})$. If standard convention is W having shape (d_{in}, d_{out}), then $Z = XW$.
Step-by-Step Solution:
Part 1: Shape of W To obtain output $(32, 50)$ from input $(32, 100)$:

$$(32, 100) \times (R, C) = (32, 50)$$

Inner dimensions must match: $R = 100$. Outer dimensions form result: $C = 50$. Shape of W is $(100, 50)$.
Part 2: Shape of b The bias is added to each sample in the batch. It must match the output dimension d_{out}. Shape of b is $(50,)$ or $(1, 50)$ for broadcasting.

Part 3: Scalar Multiplications For a matrix multiplication of $(N, M) \times (M, P)$: Operations $= N \times P \times M$.

$$\text{Ops} = 32 \times 50 \times 100 = 160,000 \text{ multiplications.}$$

Final Answer: 1. W shape: $(100, 50)$. 2. b shape: $(50,)$. 3. 160,000 multiplications.

Question 6

Question 6: Types of Learning
Match the following scenarios to the correct type of learning (Supervised, Unsupervised, Reinforcement):

1. A chess engine learns to play by playing millions of games against itself, receiving a $+1$ for a win and -1 for a loss.
2. An algorithm groups customers into 5 distinct "market segments" based on purchasing history, without knowing predefined segment labels.
3. A system predicts the price of a house based on labeled historical sales data (square footage, location, price).
4. An email filter marks messages as "Spam" or "Not Spam" based on a user-labeled dataset.

Solution

Concept:

- **Supervised:** Data has labels (Targets).
- **Unsupervised:** Data has no labels (finding patterns).
- **Reinforcement:** Agent acts in environment, learns from rewards/penalties.

Step-by-Step Solution:

1. **Chess Engine:** Uses rewards/penalties (Win/Loss) over time. $\rightarrow$ **Reinforcement Learning**.
2. **Customer Grouping:** No predefined labels, finding clusters. $\rightarrow$ **Unsupervised Learning** (Clustering).
3. **House Price:** Predicting a value from labeled examples. $\rightarrow$ **Supervised Learning** (Regression).
4. **Spam Filter:** Predicting a category from labeled examples. $\rightarrow$ **Supervised Learning** (Classification).

Question 7

Question 7: Loss Function Application
For each of the following tasks, select the most appropriate loss function from the list: [Mean Squared Error (MSE), Binary Cross-Entropy (BCE), Categorical Cross-Entropy (CCE)].

1. Predicting the temperature (in degrees Celsius) for the next day.
2. Determining whether a tumor is "Malignant" or "Benign".
3. Classifying an image of a handwritten digit into one of 10 classes (0-9).
4. Predicting the stock price of a company.

Solution

Concept:

- **Regression (Continuous output):** MSE.
- **Binary Classification (2 classes):** BCE.
- **Multi-class Classification (¿2 classes):** CCE.

Step-by-Step Solution:

1. **Temperature:** Continuous value. → **MSE**.
2. **Tumor:** Binary (Yes/No). → **BCE**.
3. **Digit (0-9):** Multi-class (10 classes). → **CCE**.
4. **Stock Price:** Continuous value. → **MSE**.

Question 8

Question 8: Training vs Inference Time
A massive deep learning model takes 2 weeks to train on a cluster of 64 GPUs.

1. Once trained, does the inference (making a prediction on a single new input) require the same computational power? Explain.
2. If the model performs a "Forward Pass" and a "Backward Pass" during training, which of these is performed during inference?
3. Why is "Gradient Descent" not used during inference?

Solution

Concept: Training involves finding parameters (weights) using optimization (Forward + Backward). Inference uses the found parameters to predict (Forward only).
Step-by-Step Solution:

1. **Power Requirement:** No. Inference is much faster and cheaper because

it only requires a single Forward pass, whereas training requires millions of Forward+Backward passes to update weights.

2. **Passes:** Only the **Forward Pass** is performed during inference. The Backward pass is for calculating gradients to update weights, which is not needed after training.

3. **Gradient Descent:** Gradient Descent is an optimization algorithm to *change* weights to minimize error. During inference, we want to *use* the fixed optimal weights, not change them.

Question 9

Question 9: Tensor Rank and Shapes
Determine the Rank (number of axes/dimensions) of the following tensors used in Deep Learning:

1. A scalar loss value $L = 0.05$.
2. A grayscale image of size 28×28.
3. A batch of 32 RGB images, each 64×64.
4. A bias vector for a layer with 10 neurons.

Solution

Concept: Rank is the number of indices required to access a specific element.

- Scalar: Rank 0.
- Vector: Rank 1.
- Matrix: Rank 2.
- 3D Tensor: Rank 3, etc.

Step-by-Step Solution:

1. **Scalar:** 0 indices needed. **Rank 0**.
2. **Grayscale Image:** (H, W). 2 indices. **Rank 2**.
3. **Batch of RGB:** $(\text{Batch}, H, W, \text{Channels})$. 4 indices. **Rank 4**.
4. **Bias Vector:** (N). 1 index. **Rank 1**.

Question 10

Question 10: Hardware Bottlenecks
You are training a model.

- GPU Computation Power: Can process 1000 images/second.

- Data Loading Speed (CPU/Disk): Can load and preprocess 800 images/second.

1. What is the effective training speed (images/second)?
2. Is this system "Compute Bound" or "I/O Bound"?
3. Suggest one specific hardware upgrade to improve speed.

Solution

Concept: The overall speed of a pipeline is limited by its slowest component (the bottleneck).
Step-by-Step Solution:

1. **Effective Speed:** The system cannot process data faster than it receives it. Thus, speed is **800 images/second**.
2. **Bottleneck Type:** Since the Computation (GPU) is waiting for Data (CPU/Disk), the system is **I/O Bound** (Input/Output Bound).
3. **Upgrade:** Improve data loading speed.
   - Replace HDD with SSD (NVMe).
   - Increase CPU cores for parallel preprocessing.

2 Module 2: Artificial Neural Networks & Perceptrons

Question 11

Question 11: Logic Gate Implementation (AND Gate)
A single perceptron with two inputs (x_1, x_2) and a bias (b) uses a step activation function:

$$y = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Determine if the following weight configuration correctly implements the logical AND gate:

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.5$$

Check the output for all 4 possible input combinations of binary inputs $\{0, 1\}$.

Solution

Concept: The AND gate outputs 1 only if both inputs are 1. Truth Table for AND:

x_1	x_2	Target t
0	0	0
0	1	0
1	0	0
1	1	1

Step-by-Step Solution: Calculate $z = w_1x_1 + w_2x_2 + b = 1 \cdot x_1 + 1 \cdot x_2 - 1.5$.

1. **Input (0, 0):** $z = 1(0) + 1(0) - 1.5 = -1.5$. Since $-1.5 < 0$, Output $y = 0$. (Correct)
2. **Input (0, 1):** $z = 1(0) + 1(1) - 1.5 = -0.5$. Since $-0.5 < 0$, Output $y = 0$. (Correct)
3. **Input (1, 0):** $z = 1(1) + 1(0) - 1.5 = -0.5$. Since $-0.5 < 0$, Output $y = 0$. (Correct)
4. **Input (1, 1):** $z = 1(1) + 1(1) - 1.5 = 0.5$. Since $0.5 \geq 0$, Output $y = 1$. (Correct)

Conclusion: Yes, the weights $(1, 1, -1.5)$ correctly implement the AND gate.

Question 12

Question 12: Designing an OR Gate
Design a single perceptron (find values for w_1, w_2, b) that implements the logical OR function. The OR gate outputs 1 if *at least one* input is 1. Use the step activation function (threshold at 0).

Solution

Concept: The OR gate separates the point $(0,0)$ (Class 0) from $(0,1),(1,0),(1,1)$ (Class 1). We need a line $w_1x_1 + w_2x_2 + b = 0$ that separates these.
Step-by-Step Solution:

1. **Constraint 1 (0,0):** $w_1(0) + w_2(0) + b < 0 \Rightarrow b < 0$.
2. **Constraint 2 (0,1):** $w_1(0) + w_2(1) + b \geq 0 \Rightarrow w_2 \geq -b$.
3. **Constraint 3 (1,0):** $w_1(1) + w_2(0) + b \geq 0 \Rightarrow w_1 \geq -b$.
4. **Selection:** Let's pick $b = -0.5$ (satisfies $b < 0$). Then $w_2 \geq 0.5$ and $w_1 \geq 0.5$. Let's pick $w_1 = 1, w_2 = 1$.
5. **Verification:** $z = x_1 + x_2 - 0.5$.
   - (0,0) $\rightarrow -0.5 \rightarrow 0$.
   - (0,1) $\rightarrow 0.5 \rightarrow 1$.
   - (1,0) $\rightarrow 0.5 \rightarrow 1$.
   - (1,1) $\rightarrow 1.5 \rightarrow 1$.

Final Answer: One possible solution is $w_1 = 1, w_2 = 1, b = -0.5$.

Question 13

Question 13: Perceptron Learning Algorithm Iteration (Part 1)
Consider a perceptron with initial weights $w = [0.5, -0.5]$ and bias $b = 0$. The learning rate is $\eta = 0.1$. Training example: Input $x = [1, 1]$, Target $t = 1$. Perform one update step of the Perceptron Learning Rule.

$$w_{new} = w_{old} + \eta(t - y)x$$

$$b_{new} = b_{old} + \eta(t - y)$$

Solution

Concept: The Perceptron Learning Rule updates weights only when the prediction is incorrect. Prediction $y = 1$ if $w \cdot x + b \geq 0$, else 0.
Step-by-Step Solution:

1. **Calculate Activation z:**
$$z = w_1x_1 + w_2x_2 + b = (0.5)(1) + (-0.5)(1) + 0 = 0.5 - 0.5 + 0 = 0.$$
2. **Determine Prediction y:** Since $z \geq 0$, output $y = 1$.
3. **Calculate Error:** Error $e = t - y = 1 - 1 = 0$.
4. **Update Weights:** Since the error is 0 (correct prediction), no update is needed.
$$w_{new} = w_{old} = [0.5, -0.5]$$

$$b_{new} = b_{old} = 0$$

Final Answer: The weights remain unchanged: $w = [0.5, -0.5], b = 0$.

Question 14

Question 14: Perceptron Learning Algorithm Iteration (Part 2)
Continuing from Question 13, consider a second training example: Input $x = [2, -1]$, Target $t = 0$. Current weights: $w = [0.5, -0.5], b = 0$. Learning rate $\eta = 0.1$. Perform the update step.

Solution

Concept: Calculate prediction, check error, apply update rule if needed.
Step-by-Step Solution:

1. **Calculate Activation z:**

$$z = w \cdot x + b = (0.5)(2) + (-0.5)(-1) + 0$$

$$z = 1.0 + 0.5 = 1.5.$$

2. **Determine Prediction y:** Since $z = 1.5 \geq 0$, output $y = 1$.

3. **Calculate Error:** Target $t = 0$. Prediction $y = 1$. Error $e = t - y = 0 - 1 = -1$. The perceptron wrongly predicted class 1.

4. **Update Weights:**

$$\Delta w = \eta \cdot e \cdot x = 0.1 \cdot (-1) \cdot [2, -1] = [-0.2, 0.1].$$

$$w_{new} = [0.5, -0.5] + [-0.2, 0.1] = [0.3, -0.4].$$

5. **Update Bias:**

$$\Delta b = \eta \cdot e = 0.1 \cdot (-1) = -0.1.$$

$$b_{new} = 0 + (-0.1) = -0.1.$$

Final Answer: New weights: $w = [0.3, -0.4]$, Bias: $b = -0.1$.

Question 15

Question 15: The XOR Problem and Linear Separability
Explain mathematically why a single perceptron cannot solve the XOR (Exclusive OR) problem. The XOR truth table is:

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0

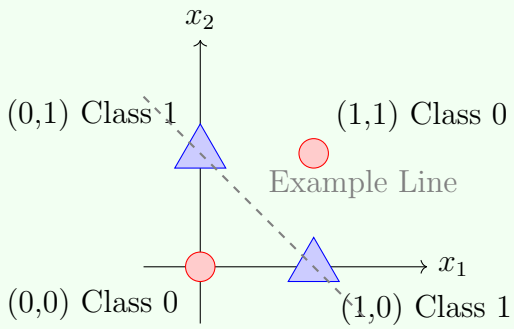
Use the concept of linear separability.

Solution

Concept: A single perceptron defines a linear decision boundary (a line in 2D): $w_1x_1 + w_2x_2 + b = 0$. For the data to be classifiable, there must exist a line that separates the points labeled '1' from those labeled '0'.

Step-by-Step Solution:

1. Plot the 4 points on a 2D plane:
   - Class 0: $(0, 0)$ and $(1, 1)$.
   - Class 1: $(0, 1)$ and $(1, 0)$.
2. **Geometric Intuition:** Imagine drawing a straight line. To separate $(0, 1)$ and $(1, 0)$ from $(0, 0)$, the line must cut between them. However, $(1, 1)$ is on the "other side" of $(0, 1)$ and $(1, 0)$ relative to $(0, 0)$. Any line separating $(0, 0)$ from the Class 1 points will put $(1, 1)$ on the Class 1 side, which is incorrect.
3. **Mathematical Contradiction:**



Let $f(x) = w_1x_1 + w_2x_2 + b$.

- For (0,0): $b < 0$ (Class 0).
- For (1,1): $w_1 + w_2 + b < 0$ (Class 0).
- For (0,1): $w_2 + b \geq 0$ (Class 1).
- For (1,0): $w_1 + b \geq 0$ (Class 1).

Adding the last two inequalities: $w_1 + w_2 + 2b \geq 0$. Subtracting the second inequality $(w_1 + w_2 + b < 0)$: $(w_1 + w_2 + 2b) - (w_1 + w_2 + b) > 0$. $b > 0$. This contradicts the first inequality $b < 0$. Thus, no such weights exist.

Final Answer: The system of inequalities required to classify XOR is inconsistent ($b < 0$ AND $b > 0$). Therefore, the data is not linearly separable and cannot be solved by a single perceptron.

Question 16

Question 16: Decision Boundary Equation
A perceptron has weights $w = [2, -1]$ and bias $b = 4$.

1. Write the equation of the decision boundary.
2. Find the x-intercept and y-intercept of this line.
3. On which side of the line is the origin $(0, 0)$ classified? (Class 0 or 1, assuming threshold ≥ 0 is Class 1).

Solution

Concept: The decision boundary is where the activation is exactly zero: $w \cdot x + b = 0$.
Step-by-Step Solution:

1. **Equation:**
$$2x_1 - 1x_2 + 4 = 0.$$
Rearranging: $x_2 = 2x_1 + 4$.
2. **Intercepts:**
   - **y-intercept ($x_1 = 0$):** $x_2 = 4$. Point $(0, 4)$.
   - **x-intercept ($x_2 = 0$):** $2x_1 + 4 = 0 \Rightarrow 2x_1 = -4 \Rightarrow x_1 = -2$. Point $(-2, 0)$.
3. **Classifying Origin $(0, 0)$:** Calculate $z = 2(0) - 1(0) + 4 = 4$. Since $4 \geq 0$, the origin is classified as **Class 1**.

Final Answer: 1. $x_2 = 2x_1 + 4$. 2. y-int: $(0, 4)$, x-int: $(-2, 0)$. 3. Class 1.

Question 17

Question 17: Convergence Theorem
The Perceptron Convergence Theorem states that if the training data is linearly separable, the perceptron learning algorithm is guaranteed to converge.

1. Does the algorithm converge if the learning rate η is very small?
2. What happens if the data is **not** linearly separable (e.g., XOR)?
3. Does the algorithm stop automatically in the non-separable case?

Solution

Concept: Convergence depends on the existence of a solution.

1. **Small η:** Yes, it still converges, but it might take more steps. The theorem holds for any $\eta > 0$.
2. **Non-separable:** The algorithm will loop forever, oscillating between dif-

ferent weight vectors. It will never find a set of weights that classifies all examples correctly.

3. **Stopping:** No, it does not stop automatically. A maximum number of epochs (iterations) must be set as a stopping criterion.

Question 18

Question 18: Bias vs Threshold
Some perceptron formulations use a threshold T instead of a bias b.

$$y = 1 \text{ if } w \cdot x \geq T, \text{ else } 0$$

Show how this is mathematically equivalent to the standard form $w \cdot x + b \geq 0$. If a perceptron has threshold $T = 5$, what is the equivalent bias b?

Solution

Concept: Rearrange the inequality.
Step-by-Step Solution:

1. **Formulation:** Original: $w \cdot x \geq T$. Subtract T from both sides: $w \cdot x - T \geq 0$.
2. **Equivalence:** Compare with $w \cdot x + b \geq 0$. We see that $b = -T$.
3. **Calculation:** If $T = 5$, then $b = -5$.

Final Answer: The bias is the negative of the threshold. Equivalent bias $b = -5$.

Question 19

Question 19: Multilayer Perceptron (MLP) Concept
A single perceptron fails XOR. However, by combining perceptrons, we can solve it. Consider a network with 2 hidden neurons (h_1, h_2) and 1 output neuron (y). Let h_1 act as an OR gate. Let h_2 act as a NAND gate (NOT AND). Let y act as an AND gate taking inputs from h_1 and h_2. Does this network solve XOR? Prove by Truth Table.

Solution

Concept: XOR is true if (A OR B) is true AND (A NAND B) is true. $A \oplus B = (A \vee B) \wedge \neg(A \wedge B)$.
Step-by-Step Solution: Truth Table:

A	B	h_1 (OR)	h_2 (NAND)	y (h_1 AND h_2)
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

The column for y is $[0, 1, 1, 0]$, which matches the XOR truth table. **Conclusion:** Yes, this 2-layer architecture solves XOR.

Question 20

Question 20: Hyperplanes in Higher Dimensions
A perceptron has 3 inputs (x_1, x_2, x_3) and weights (w_1, w_2, w_3).

1. What is the geometric shape of the decision boundary in this 3D input space?
2. If $w = [1, 1, 1]$ and $b = 0$, does the point $(1, -1, -1)$ belong to the positive class?

Solution

Concept: In n-dimensions, a linear equation defines a hyperplane of dimension $n - 1$.
Step-by-Step Solution:

1. **Shape:** In 3D space, the boundary is a 2D **Plane**.
2. **Classification:** Calculate dot product:

$$z = 1(1) + 1(-1) + 1(-1) + 0$$

$$z = 1 - 1 - 1 = -1.$$

Since $-1 < 0$, it belongs to the **Negative Class (0)**.

3 Module 3: Linear Neural Networks for Regression

Question 21

Question 21: Linear Regression Prediction
A simple linear regression model predicts the price of a house (y) based on its size in square feet (x_1) and number of bedrooms (x_2). The weights are $w_1 = 0.5$, $w_2 = 20$, and bias $b = 100$. (Units: Price in \$1000s, Size in sq ft, Bedrooms count). Calculate the predicted price for a house with 2000 sq ft and 3 bedrooms.

Solution

Concept: The linear model equation is:

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

Step-by-Step Solution:

1. **Given:** $x_1 = 2000$, $x_2 = 3$. $w_1 = 0.5$, $w_2 = 20$, $b = 100$.
2. **Substitute values:**

$$\hat{y} = (0.5 \times 2000) + (20 \times 3) + 100$$

$$\hat{y} = 1000 + 60 + 100$$

$$\hat{y} = 1160$$

3. **Interpretation:** The predicted price is \$1,160,000.

Final Answer: $\hat{y} = 1160$ (\$1.16M).

Question 22

Question 22: Mean Squared Error (MSE) Calculation
Consider a dataset with three points: $A : (x = 1, y = 2)$, $B : (x = 2, y = 4)$, $C : (x = 3, y = 5)$. A model predicts $\hat{y} = 1.5x + 0.5$. Calculate the Mean Squared Error (MSE) for this model on this dataset.

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Solution

Concept: MSE measures the average squared difference between predicted and actual values.
Step-by-Step Solution:

1. **Point A (1, 2):** $\hat{y}_A = 1.5(1) + 0.5 = 2.0$. Error $e_A = 2 - 2 = 0$. Squared Error $e_A^2 = 0$.
2. **Point B (2, 4):** $\hat{y}_B = 1.5(2) + 0.5 = 3.5$. Error $e_B = 4 - 3.5 = 0.5$. Squared

Error $e_B^2 = 0.25$.

3. **Point C (3, 5):** $\hat{y}_C = 1.5(3) + 0.5 = 5.0$. Error $e_C = 5 - 5 = 0$. Squared Error $e_C^2 = 0$.

4. **Average:** Sum of Squared Errors $= 0 + 0.25 + 0 = 0.25$. $N = 3$.

$$\text{MSE} = \frac{0.25}{3} \approx 0.0833.$$

Final Answer: MSE ≈ 0.0833.

Question 23

Question 23: Gradient Calculation (Single Weight)
For the regression model $\hat{y} = wx$ (no bias), the loss function for a single example is:

$$L(w) = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(y - wx)^2$$

Calculate the gradient of the loss with respect to w ($\frac{\partial L}{\partial w}$) for the data point ($x = 2, y = 10$) when the current weight is $w = 4$.

Solution

Concept: Chain rule derivative:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

$$\frac{\partial L}{\partial \hat{y}} = -(y - \hat{y})$$

$$\frac{\partial \hat{y}}{\partial w} = x$$

$$\frac{\partial L}{\partial w} = -(y - \hat{y})x$$

Step-by-Step Solution:

1. **Calculate Prediction $\hat{y}$:** $w = 4, x = 2 \Rightarrow \hat{y} = 8$.

2. **Calculate Error Term:** $y - \hat{y} = 10 - 8 = 2$.

3. **Calculate Gradient:** Gradient $= -(2) \times 2 = -4$.

Final Answer: Gradient $\frac{\partial L}{\partial w} = -4$.

Question 24

Question 24: Gradient Descent Update (One Step)
Using the result from Question 23 (Gradient = -4), perform one update step for the weight w using Gradient Descent. Assume a learning rate $\eta = 0.1$. Current

weight $w_{old} = 4$.

$$w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$$

Solution

Concept: Gradient Descent subtracts a fraction of the gradient to move towards the minimum.
Step-by-Step Solution:

1. **Update Rule:**
$$w_{new} = 4 - (0.1 \times (-4))$$
$$w_{new} = 4 - (-0.4)$$
$$w_{new} = 4 + 0.4 = 4.4$$

2. **Intuition:** The gradient was negative, meaning increasing w decreases Loss. The update correctly increased w from 4 to 4.4 (closer to ideal $w = 5$ since $y = 10, x = 2 \Rightarrow w = 5$).

Final Answer: New weight $w_{new} = 4.4$.

Question 25

Question 25: Stochastic Gradient Descent (SGD) vs Batch GD
You have a dataset of 1,000,000 examples.

1. To perform one update step in Batch Gradient Descent, how many examples must be processed?

2. To perform one update step in Stochastic Gradient Descent (SGD), how many examples must be processed?

3. Which method is noisier? Explain why.

Solution

Concept: Batch GD uses the entire dataset for the true gradient. SGD uses a single random example to estimate the gradient.
Step-by-Step Solution:

1. **Batch GD:** Must process all $1,000,000$ examples to compute the sum of gradients before updating.

2. **SGD:** Processes only 1 example to compute an approximate gradient and update immediately.

3. **Noise:** SGD is noisier because the gradient from a single example may not point in the exact direction of the global minimum (it fluctuates). However, this noise can help escape local minima.

Final Answer: 1. 1,000,000. 2. 1. 3. SGD is noisier.

Question 26

Question 26: Convexity of MSE
Mathematically demonstrate that the Mean Squared Error function $L(w) = (y - wx)^2$ is convex with respect to w. Hint: Check the second derivative.

Solution

Concept: A function is convex if its second derivative is non-negative everywhere ($\frac{\partial^2 L}{\partial w^2} \geq 0$).
Step-by-Step Solution:

1. **First Derivative:**

$$\frac{\partial L}{\partial w} = 2(y - wx)(-x) = -2x(y - wx).$$

2. **Second Derivative:** Differentiate $-2xy + 2wx^2$ with respect to w:

$$\frac{\partial^2 L}{\partial w^2} = \frac{\partial}{\partial w}(2x^2 w - 2xy) = 2x^2.$$

3. **Analysis:** Since x^2 is always non-negative for any real number x, $2x^2 \geq 0$. Therefore, the function is convex.

Final Answer: The second derivative $2x^2 \geq 0$, proving convexity.

Question 27

Question 27: Feature Scaling (Normalization)
You are training a linear regression model on a dataset with two features:

- x_1: Age (Range 0-100).
- x_2: Income (Range 0-1,000,000).

1. Why might Gradient Descent converge slowly on this unscaled data?
2. Calculate the standardized value (z-score) for an income of 50,000 if the mean is 40,000 and standard deviation is 20,000.

Solution

Concept: Unscaled features create "elongated" loss surfaces (valleys), causing GD to oscillate. Standardization ($z = \frac{x-\mu}{\sigma}$) makes the contours spherical, enabling faster convergence.
Step-by-Step Solution:

1. **Convergence Issue:** The gradients for w_2 (Income) will be massive compared to w_1 (Age), forcing a very small learning rate. This makes updates for w_1 tiny, slowing down convergence.

2. **Standardization Calculation:**

$$z = \frac{x - \mu}{\sigma}$$

$$z = \frac{50,000 - 40,000}{20,000} = \frac{10,000}{20,000} = 0.5.$$

Final Answer: 1. Elongated loss surface causes oscillation. 2. $z = 0.5$.

Question 28

Question 28: Learning Rate Divergence
Consider the function $L(w) = w^2$. The minimum is at $w = 0$. Current $w = 2$. Gradient $\frac{\partial L}{\partial w} = 2w = 4$. Perform one update step with a large learning rate $\eta = 1.1$. What is the new w? Did we get closer to 0?

Solution

Concept: If η is too large, the update can overshoot the minimum and diverge.
Step-by-Step Solution:

1. **Update:**

$$w_{new} = w_{old} - \eta \cdot \text{Gradient}$$

$$w_{new} = 2 - 1.1(4)$$

$$w_{new} = 2 - 4.4 = -2.4.$$

2. **Comparison:** Old distance from minimum (0): $|2| = 2$. New distance from minimum (0): $|-2.4| = 2.4$.

3. **Result:** The distance increased ($2.4 > 2$). The parameters are diverging.

Final Answer: $w_{new} = -2.4$. We moved further away from the minimum.

Question 29

Question 29: Polynomial Regression via Linear Models
A dataset follows a parabolic trend $y \approx x^2$. Can a linear model $y = wx + b$ fit this data well? How can we modify the *features* (not the model) to fit this parabola using linear regression?

Solution

Concept: Linear regression is linear in *parameters* (w), not necessarily in input features (x). We can engineer non-linear features.
Step-by-Step Solution:

1. **Linear Model Fit:** No, a straight line cannot fit a parabola well (high bias/underfitting).

2. **Feature Engineering:** Create a new feature $x_{new} = x^2$. Now train a linear

model:

$$y = w_1 x_{new} + b \Rightarrow y = w_1(x^2) + b.$$

This is still a linear regression problem (linear in w_1), but it models a quadratic relationship with respect to the original x.

Question 30

Question 30: The Normal Equation (Closed Form)
Instead of Gradient Descent, we can solve for weights analytically using the Normal Equation:

$$w = (X^T X)^{-1} X^T y$$

Given $X = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ (a column vector of inputs) and $y = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$. Calculate $(X^T X)$ and its inverse.

Solution

Concept: Matrix operations for least squares solution.
Step-by-Step Solution:

1. **Compute $X^T X$:**

$$X^T = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = (1)(1) + (2)(2) = 1 + 4 = 5.$$

   This is a scalar (1x1 matrix).

2. **Compute Inverse $(X^T X)^{-1}$:** The inverse of scalar 5 is $1/5 = 0.2$.

Final Answer: $X^T X = 5$. Inverse $= 0.2$.

4 Module 4: Linear Neural Networks for Classification

Question 31

Question 31: Sigmoid Activation Function
A neuron in a binary classification network outputs a raw score (logit) $z = 0$. Calculate the output probability $\hat{y} = \sigma(z)$ using the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Solution

Concept: The sigmoid function squashes any real number into the range $(0, 1)$, representing a probability.
Step-by-Step Solution:

1. **Substitute $z = 0$:**
$$\sigma(0) = \frac{1}{1 + e^{-0}}$$
2. **Evaluate Exponent:** $e^0 = 1$.
3. **Calculate Result:**
$$\sigma(0) = \frac{1}{1 + 1} = \frac{1}{2} = 0.5.$$

Final Answer: $\hat{y} = 0.5$ (50% probability).

Question 32

Question 32: Binary Cross-Entropy (BCE) Loss
In a binary classification task, for a single example with true label $y = 1$, the model predicts a probability $\hat{y} = 0.9$. Calculate the Binary Cross-Entropy loss:

$$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

(Use $\ln(0.9) \approx -0.105$).

Solution

Concept: BCE penalizes wrong confident predictions heavily. Since $y = 1$, only the first term matters.
Step-by-Step Solution:

1. **Substitute values:** $y = 1$, $\hat{y} = 0.9$.
$$L = -[1 \cdot \ln(0.9) + (1 - 1) \cdot \ln(1 - 0.9)]$$
$$L = -[\ln(0.9) + 0]$$
2. **Calculate Log:** $\ln(0.9) \approx -0.105$.

3. **Result:**

$$L = -(-0.105) = 0.105.$$

Final Answer: Loss $L \approx 0.105$.

Question 33

Question 33: Log-Odds (Logits) Interpretation
The output of a logistic regression model is a probability $\hat{y} = 0.8$. Calculate the log-odds (logit) z corresponding to this probability. Hint: The inverse of the sigmoid function is the logit function:

$$z = \ln\left(\frac{\hat{y}}{1-\hat{y}}\right)$$

Solution

Concept: The logit z represents the log of the odds ratio (Probability of success / Probability of failure).
Step-by-Step Solution:

1. **Calculate Odds Ratio:**

$$\text{Odds} = \frac{0.8}{1-0.8} = \frac{0.8}{0.2} = 4.$$

2. **Calculate Logit:** $z = \ln(4)$. Using approximation $\ln(4) \approx 1.386$.

Final Answer: Logit $z \approx 1.386$.

Question 34

Question 34: Decision Boundary from Weights
A logistic regression model has weights $w_1 = 2$, $w_2 = -3$, and bias $b = 1$. The decision boundary is the line where the probability is 0.5 (i.e., $z = 0$). Determine the equation of the decision boundary line in the form $x_2 = mx_1 + c$.

Solution

Concept: At the decision boundary, $w \cdot x + b = 0$.
Step-by-Step Solution:

1. **Set Logit to 0:**

$$2x_1 - 3x_2 + 1 = 0.$$

2. **Rearrange for x_2:**

$$3x_2 = 2x_1 + 1$$
$$x_2 = \frac{2}{3}x_1 + \frac{1}{3}.$$

3. **Result:** Slope $m = 2/3$, Intercept $c = 1/3$.

Final Answer: $x_2 = \frac{2}{3}x_1 + \frac{1}{3}$.

Question 35

Question 35: Comprehensive Classification Metrics
A medical diagnostic system for detecting a rare disease produces the following confusion matrix on a test set of 200 patients:

	Predicted Positive	Predicted Negative
Actual Positive	80 (TP)	10 (FN)
Actual Negative	20 (FP)	90 (TN)

Calculate the following metrics (show formulas and steps):

1. **Precision:** (Positive Predictive Value)
2. **Recall:** (Sensitivity / True Positive Rate)
3. **Specificity:** (True Negative Rate)
4. **F1-Score:** (Harmonic Mean of Precision and Recall)

Solution

Concept:

- Precision $= \frac{TP}{TP+FP}$ (Quality of positive predictions).
- Recall $= \frac{TP}{TP+FN}$ (Quantity of positives found).
- Specificity $= \frac{TN}{TN+FP}$ (Ability to find negatives).
- F1 $= 2 \cdot \frac{P \cdot R}{P+R}$.

Step-by-Step Solution:

1. Precision

$$\text{Precision} = \frac{80}{80+20} = \frac{80}{100} = 0.80$$

2. Recall (Sensitivity)

$$\text{Recall} = \frac{80}{80+10} = \frac{80}{90} \approx 0.8889$$

3. Specificity

$$\text{Specificity} = \frac{90}{90+20} = \frac{90}{110} \approx 0.8182$$

4. F1-Score

$$\text{F1} = 2 \cdot \frac{0.80 \cdot 0.8889}{0.80 + 0.8889}$$

$$\text{Numerator} = 0.7111$$

$$\text{Denominator} = 1.6889$$

$$\text{F1} = 2 \cdot 0.421 = 0.842$$

Final Answer: Precision: 0.80, Recall: 0.89, Specificity: 0.82, F1: 0.84.

Question 36

Question 36: ROC Curve and AUC
Two models, A and B, are evaluated on the same binary classification dataset.

- Model A has an Area Under the ROC Curve (AUC) of 0.95.
- Model B has an AUC of 0.50.

1. Which model is better?
2. What behavior does an AUC of 0.50 represent?
3. If a model has an AUC of 0.10, is it useless? Explain.

Solution

Concept: ROC measures the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) at various thresholds. AUC quantifies the overall performance.
Step-by-Step Solution:

1. **Comparison:** Model A (0.95) is far better. An AUC of 1.0 represents a perfect classifier.
2. **AUC 0.50:** This represents a **Random Classifier**. The model has no discriminative power; it is guessing (like flipping a coin).
3. **AUC 0.10:** No, it is not useless. It is actually a **good classifier that is inverted**. If you flip its predictions ($0 \to 1, 1 \to 0$), the AUC becomes $1 - 0.10 = 0.90$, which is excellent.

Final Answer: 1. Model A. 2. Random guessing. 3. No, it's inverted (good predictor if flipped).

Question 37

Question 37: Class Imbalance and Accuracy Trap
A dataset has 9900 samples of Class 0 (Benign) and 100 samples of Class 1 (Malignant). A "dumb" model predicts Class 0 for every single input.

1. Calculate the Accuracy of this model.
2. Why is Accuracy a misleading metric here?
3. Which metric would reveal the model's failure?

Solution

Concept: Accuracy $= \frac{\text{Correct Predictions}}{\text{Total Predictions}}$.
Step-by-Step Solution:

1. **Accuracy Calculation:** Correct (Class 0) = 9900. Wrong (Class 1) = 100.

$$\text{Accuracy} = \frac{9900}{10000} = 0.99 \ (99\%).$$

2. **Misleading Nature:** The model appears excellent (99% accuracy) but completely fails to detect the critical minority class (0% detection of Malignant cases).

3. **Better Metric: Recall** (Sensitivity) for Class 1. Recall $= \frac{TP}{TP+FN} = \frac{0}{0+100} = 0$. This correctly shows the model is failing.

Final Answer: 1. 99%. 2. Fails to detect minority class. 3. Recall.

Question 38

Question 38: Softmax Probability (Multi-class)
A neural network outputs three raw logits for a 3-class problem: $z = [2.0, 1.0, 0.1]$. Calculate the probability for the first class ($P(y = 0)$) using the Softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

(Use $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^{0.1} \approx 1.11$).

Solution

Concept: Softmax normalizes logits into a probability distribution summing to 1.
Step-by-Step Solution:

1. **Calculate Exponentials:** $e^{2.0} \approx 7.39$. $e^{1.0} \approx 2.72$. $e^{0.1} \approx 1.11$.

2. **Calculate Sum:** Sum $= 7.39 + 2.72 + 1.11 = 11.22$.

3. **Calculate Probability for Class 0:**

$$P(y = 0) = \frac{7.39}{11.22} \approx 0.659.$$

Final Answer: $P(y = 0) \approx 0.66$ (66%).

Question 39

Question 39: Gradient of the Sigmoid Function
Show that the derivative of the sigmoid function $\sigma(z)$ with respect to z can be expressed in terms of $\sigma(z)$ itself:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

This property is crucial for efficient backpropagation.

Solution

Concept: Use the quotient rule or chain rule on $\sigma(z) = (1 + e^{-z})^{-1}$.
Step-by-Step Solution:

1. **Differentiate:**

$$\frac{d}{dz}(1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

2. **Rearrange:**

$$\frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}.$$

3. **Substitute $\sigma(z)$:** The first term is $\sigma(z)$. The second term is:

$$\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}} = 1 - \sigma(z).$$

4. **Combine:**

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Question 40

Question 40: Mini-Batch Effects
Why is Mini-Batch Gradient Descent preferred over Stochastic Gradient Descent (Batch Size = 1) for training deep networks? Focus on computational efficiency and stability.

Solution

Concept: Trade-off between noise and speed.
Step-by-Step Solution:

1. **Vectorization:** GPUs are optimized for matrix operations. Processing a batch (e.g., 32 images) simultaneously is much faster than processing 32 images sequentially (SGD) due to parallelization.

2. **Stability:** The gradient estimated from a batch has lower variance (less noise) than a single example, leading to smoother convergence while still allowing some noise to escape local minima.

5 Module 5: Deep Feed-Forward Neural Networks (DFNN)

Question 41

Question 41: MLP Memory Footprint Calculation
Consider a Multi-Layer Perceptron (MLP) used for image classification:

- Input: 784 floats (Image).
- Layer 1: 1024 neurons.
- Layer 2: 512 neurons.
- Output: 10 neurons.

Assume all values (weights, biases, activations, gradients) are stored as 32-bit floats (4 bytes). Calculate the memory required in Megabytes (MB) for:

1. Storing the Model Parameters (Weights + Biases).
2. Storing the Activations for a single Batch of size $B = 64$ (Forward Pass memory).
3. Storing the Gradients for the Parameters (Backward Pass memory).

Solution

Concept: Memory = Count $\times$ 4 bytes. 1 MB = 1024^2 bytes (approx 10^6).
Step-by-Step Solution:
Part 1: Model Parameters

- L1: $(784 \times 1024) + 1024 = 802,816 + 1,024 = 803,840$.
- L2: $(1024 \times 512) + 512 = 524,288 + 512 = 524,800$.
- Out: $(512 \times 10) + 10 = 5,120 + 10 = 5,130$.
- Total Params $P = 1,333,770$.
- Memory $= 1,333,770 \times 4$ bytes ≈ 5.34 MB.

Part 2: Activations (Batch $B = 64$) We store output of each layer for backprop (excluding input usually, but let's count hidden layers).

- L1 Output: $64 \times 1024 = 65,536$.
- L2 Output: $64 \times 512 = 32,768$.
- Out Output: $64 \times 10 = 640$.
- Total Values $= 98,944$.
- Memory $= 98,944 \times 4$ bytes ≈ 0.39 MB.

Part 3: Gradients Gradients have the exact same shape as parameters.

- Memory = Same as Params $\approx$ 5.34 MB.

Final Answer: 1. Params: 5.34 MB. 2. Activations (Batch 64): 0.39 MB. 3. Gradients: 5.34 MB.

Question 42

Question 42: Forward Pass Calculation
Consider a simple network with 1 hidden layer (2 neurons) and 1 output neuron. Inputs: $x = [1, 2]$. Weights (Input to Hidden): $W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1.0 & 0.0 \end{bmatrix}$ (Rows are destination neurons). Bias (Hidden): $b^{(1)} = [0, 0]$. Weights (Hidden to Output): $W^{(2)} = [1, -1]$. Bias (Output): $b^{(2)} = 0$. Activation: ReLU ($f(z) = \max(0, z)$). Calculate the final output.

Solution

Concept: Compute linear combination $\rightarrow$ Apply Activation $\rightarrow$ Repeat.
Step-by-Step Solution:

1. **Hidden Layer Pre-activation ($z^{(1)}$):**

$$z_1 = (0.5)(1) + (-0.5)(2) + 0 = 0.5 - 1.0 = -0.5.$$

$$z_2 = (1.0)(1) + (0.0)(2) + 0 = 1.0.$$

$z^{(1)} = [-0.5, 1.0]$.

2. **Hidden Layer Activation ($h^{(1)}$):** ReLU applied element-wise:

$$h_1 = \max(0, -0.5) = 0.$$

$$h_2 = \max(0, 1.0) = 1.0.$$

$h^{(1)} = [0, 1.0]$.

3. **Output Layer Pre-activation ($z^{(2)}$):**

$$z^{(2)} = W^{(2)} \cdot h^{(1)} + b^{(2)}.$$

$$z^{(2)} = (1)(0) + (-1)(1.0) + 0 = -1.0.$$

4. **Output Layer Activation (y):** Usually output layer depends on task (e.g., Sigmoid for binary). Let's assume linear or ReLU. If Linear: $y = -1.0$. If ReLU: $y = 0$. Let's assume Linear (Regression).

Final Answer: Output $y = -1.0$ (Linear activation).

Question 43

Question 43: ReLU vs Sigmoid Gradients
Calculate the derivative of the ReLU function and the Sigmoid function at $z = 4$. Which one is better for deep networks and why?

Solution

Concept: Gradient magnitude affects learning speed (vanishing gradient problem).
Step-by-Step Solution:

1. **ReLU Derivative:** $f(z) = \max(0, z)$. $f'(z) = 1$ if $z > 0$, else 0. At $z = 4$, $f'(4) = 1$.
2. **Sigmoid Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. $\sigma(4) \approx 0.982$. $\sigma'(4) \approx 0.982(1 - 0.982) \approx 0.982(0.018) \approx 0.017$.
3. **Comparison:** The ReLU gradient is 1 (strong signal). The Sigmoid gradient is ≈ 0.017 (very weak signal). Multiplying many small gradients (Chain Rule) causes them to vanish to zero.

Final Answer: ReLU is better because its gradient does not vanish for positive inputs, enabling training of deeper networks.

Question 44

Question 44: Vanishing Gradient Problem
In a deep network with 4 hidden layers using Sigmoid activation, the gradients at each layer are approximately 0.25. Estimate the magnitude of the gradient at the first layer (relative to the output gradient).

Solution

Concept: Chain rule multiplies gradients.

$$\text{Total Gradient} \propto \prod(\text{Local Gradients})$$

Step-by-Step Solution:

1. **Multiply Gradients:** $0.25 \times 0.25 \times 0.25 \times 0.25 = (0.25)^4$.
2. **Calculation:** $(1/4)^4 = 1/256 \approx 0.0039$.
3. **Interpretation:** The gradient reaching the first layer is less than 0.4% of the original signal, making learning extremely slow.

Final Answer: ≈ 0.0039.

Question 45

Question 45: Backpropagation (Chain Rule Application)
Given a computational graph: $x \to z \to h \to L$. Equations: $z = wx$. $h = \sigma(z)$.

$L = (t - h)^2$. Write the expression for $\frac{\partial L}{\partial w}$ using the Chain Rule.

Solution

Concept: Multiply partial derivatives from Output back to Input.
Step-by-Step Solution:

1. **Decompose Path:**
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial w}.$$
2. **Compute Components:** $\frac{\partial L}{\partial h} = -2(t - h)$. $\frac{\partial h}{\partial z} = \sigma'(z)$. $\frac{\partial z}{\partial w} = x$.
3. **Combine:**
$$\frac{\partial L}{\partial w} = -2(t - h) \cdot \sigma'(z) \cdot x.$$

Question 46

Question 46: Universal Approximation Theorem
Does the Universal Approximation Theorem state that a neural network can learn *any* function perfectly? What are the conditions (width/depth)?

Solution

Concept: Theoretical capability vs Trainability.
Step-by-Step Solution:

1. **Theorem:** A feed-forward network with a *single* hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of $\mathbb{R}^n$, under mild assumptions on the activation function.
2. **Constraint:** The layer might need to be arbitrarily *wide* (exponential number of neurons).
3. **Answer:** No, it doesn't say it can "learn" it (optimization might fail), but it can "represent" it. And it requires sufficient width.

Question 47

Question 47: Weight Initialization
Why should weights in a neural network **not** be initialized to zero? What happens during backpropagation if they are?

Solution

Concept: Symmetry breaking.
Step-by-Step Solution:

1. **Forward Pass:** If all weights are 0, all neurons in a layer receive the same input (0) and output the same value (assuming bias is same).

2. **Backward Pass:** Since all neurons output the same value, they all receive the same gradient update.

3. **Result:** All neurons remain identical throughout training. The network acts like a single neuron. We need random initialization to break this symmetry.

Question 48

Question 48: Leaky ReLU Calculation
The Leaky ReLU function is defined as:

$$f(z) = \max(0.01z, z)$$

Calculate the output for input $z = -10$. How does this solve the "Dying ReLU" problem?

Solution

Concept: Allowing a small gradient for negative inputs keeps neurons alive.
Step-by-Step Solution:

1. **Calculation:** Since $z < 0$, $f(z) = 0.01 \times (-10) = -0.1$.

2. **Dying ReLU:** Standard ReLU outputs 0 for negative inputs, meaning gradient is 0. If a neuron is always negative, it stops learning (dies). Leaky ReLU provides a small non-zero gradient (0.01), allowing recovery.

Final Answer: $y = -0.1$.

Question 49

Question 49: Output Layer Activation Choice
Select the correct activation function for the output layer for the following tasks:

1. Predicting House Price (positive real number).

2. Sentiment Analysis (Positive/Negative).

3. Image Classification (Cat, Dog, Bird).

Solution

Concept: Match range of function to range of targets.
Step-by-Step Solution:

1. **Regression (Positive):** ReLU (outputs $[0, \infty)$) or Linear (if standardized). Usually Linear is standard for regression.

2. **Binary Classification:** Sigmoid (outputs $[0, 1]$ probability).

3. **Multi-class Classification:** Softmax (outputs probability distribution).

Question 50

Question 50: Depth vs Width
Which architecture is generally more parameter-efficient for learning complex hierarchical features: Deep (many layers) or Wide (one huge layer)? Explain using the example of an image (Pixels → Edges → Shapes → Objects).

Solution

Concept: Hierarchical composition.
Step-by-Step Solution:

1. **Efficiency:** Deep networks are exponentially more efficient.
2. **Reasoning:** Features like "Edges" can be reused to form many different "Shapes". "Shapes" can be reused to form "Objects". A Deep network learns these shared compositions. A Wide network (1 layer) must memorize every possible combination of pixels for every object independently, requiring exponentially more units.

6 Module 6: Convolutional Neural Networks (CNN)

Question 51

Question 51: Output Feature Map Dimension Calculation
A convolutional layer has the following specifications:

- Input Volume: $W_{in} = 32 \times 32 \times 3$ (RGB Image).
- Filter Size (F): 5×5.
- Stride (S): 1.
- Padding (P): 0 (Valid Padding).
- Number of Filters (K): 10.

Calculate the spatial dimensions ($H_{out} \times W_{out}$) and the depth (D_{out}) of the output feature map. Formula: $W_{out} = \lfloor \frac{W_{in}-F+2P}{S} \rfloor + 1$.

Solution

Concept: The output size depends on how many times the filter can slide across the input. The depth is equal to the number of filters.
Step-by-Step Solution:

1. **Identify Parameters:** $W_{in} = 32$, $F = 5$, $S = 1$, $P = 0$.
2. **Apply Formula:**
$$W_{out} = \frac{32 - 5 + 2(0)}{1} + 1$$
$$W_{out} = \frac{27}{1} + 1 = 28.$$
Since $H_{in} = W_{in}$ and F is square, $H_{out} = 28$.
3. **Determine Depth:** $D_{out} = K = 10$.

Final Answer: Output dimensions: $28 \times 28 \times 10$.

Question 52

Question 52: Network-in-Network (NiN) Block Analysis
Consider a simplified Network-in-Network (NiN) block consisting of three convolutional layers sequentially applied to a 256×256 grayscale input image ($C_{in} = 1$):

- **Layer 1:** 1×1 convolution with 20 filters.
- **Layer 2:** 3×3 convolution with 50 filters, stride 1, and "Same" padding.
- **Layer 3:** 1×1 convolution with 20 filters.

Assume ReLU activation follows each convolution. Calculate:

1. The number of parameters (weights + biases) for each layer individually.

2. The spatial dimensions ($H \times W \times C$) of the feature map after each layer.
3. The total number of learnable parameters in the entire NiN block.

Solution

Concept:

- **1x1 Conv Params:** $(1 \times 1 \times C_{in} \times K) + K$ (bias).
- **3x3 Conv Params:** $(3 \times 3 \times C_{in} \times K) + K$ (bias).
- **Output Dimensions:** For 1×1 or "Same" padding with stride 1, spatial dimensions (H, W) remain unchanged; only depth (C) changes to K.

Step-by-Step Solution:
Part 1: Layer-wise Parameters

- **Layer 1 (1×1, $C_{in} = 1, K = 20$):** Weights: $1 \times 1 \times 1 \times 20 = 20$. Biases: 20. Total: $20 + 20 = 40$.
- **Layer 2 (3×3, $C_{in} = 20, K = 50$):** Input depth comes from Layer 1 output (20). Weights: $3 \times 3 \times 20 \times 50 = 9 \times 1000 = 9000$. Biases: 50. Total: $9000 + 50 = 9050$.
- **Layer 3 (1×1, $C_{in} = 50, K = 20$):** Input depth comes from Layer 2 output (50). Weights: $1 \times 1 \times 50 \times 20 = 1000$. Biases: 20. Total: $1000 + 20 = 1020$.

Part 2: Feature Map Dimensions

- **Input:** $256 \times 256 \times 1$.
- **After Layer 1:** Spatial unchanged (1×1), Depth becomes 20. $\rightarrow$ $\mathbf{256 \times 256 \times 20}$.
- **After Layer 2:** Spatial unchanged (Same padding), Depth becomes 50. $\rightarrow$ $\mathbf{256 \times 256 \times 50}$.
- **After Layer 3:** Spatial unchanged (1×1), Depth becomes 20. $\rightarrow$ $\mathbf{256 \times 256 \times 20}$.

Part 3: Total Parameters

$$40 + 9050 + 1020 = 10,110.$$

Final Answer: 1. L1: 40, L2: 9050, L3: 1020. 2. Output final shape: $256 \times 256 \times 20$. 3. Total: 10,110 parameters.

Question 53

Question 53: Padding Calculation for "Same" Output
You have an input of size 28×28. You want to apply a 3×3 convolution with stride

$S = 1$ such that the output size remains 28×28 ("Same" padding). Calculate the required padding P.

Solution

Concept: For "Same" padding with $S = 1$, we want $W_{out} = W_{in}$.

$$W_{in} = \frac{W_{in} - F + 2P}{1} + 1$$

$$0 = -F + 2P + 1 \Rightarrow 2P = F - 1 \Rightarrow P = \frac{F-1}{2}.$$

Step-by-Step Solution:

1. **Substitute $F = 3$:**
$$P = \frac{3-1}{2} = \frac{2}{2} = 1.$$

Final Answer: Padding $P = 1$.

Question 54

Question 54: Pooling Layer Output
Consider a 4×4 matrix:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 8 & 3 & 0 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$

Apply Max Pooling with a 2×2 filter and stride $S = 2$. Draw the output matrix.

Solution

Concept: Divide the input into non-overlapping 2×2 regions (since $S = 2$) and take the maximum value from each region.
Step-by-Step Solution:

1. **Region 1 (Top-Left):** $\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} \to \max(1, 3, 5, 6) = 6.$
2. **Region 2 (Top-Right):** $\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix} \to \max(2, 4, 1, 2) = 4.$
3. **Region 3 (Bottom-Left):** $\begin{bmatrix} 7 & 8 \\ 2 & 1 \end{bmatrix} \to \max(7, 8, 2, 1) = 8.$
4. **Region 4 (Bottom-Right):** $\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \to \max(3, 0, 0, 1) = 3.$
5. **Construct Output:**

$$\begin{array}{|c|c|c|c|}\hline 1 & 3 & 2 & 4 \\ \hline 5 & 6 & 1 & 2 \\ \hline 7 & 8 & 3 & 0 \\ \hline 2 & 1 & 0 & 1 \\ \hline \end{array} \xrightarrow{\text{Max Pool}} \begin{array}{|c|c|}\hline 6 & 4 \\ \hline 8 & 3 \\ \hline \end{array}$$

$$\begin{bmatrix} 6 & 4 \\ 8 & 3 \end{bmatrix}$$

Final Answer: A 2×2 matrix: $\begin{bmatrix} 6 & 4 \\ 8 & 3 \end{bmatrix}$.

Question 55

Question 55: 1x1 Convolution (Dimensionality Reduction)
An input volume has dimensions $14 \times 14 \times 256$ (e.g., in a ResNet block). We apply a 1×1 convolution layer with 64 filters.

1. What is the output volume dimension?
2. How does this operation reduce computational cost for subsequent layers?

Solution

Concept: 1×1 convolution changes the depth (C) without affecting spatial dimensions (H, W). It is used to reduce the number of channels.
Step-by-Step Solution:

1. **Dimensions:** Spatial: 14×14 (unchanged). Depth: Becomes equal to the number of filters (64). Output: $14 \times 14 \times 64$.
2. **Cost Reduction:** The number of channels decreased from 256 to 64 (4x reduction). Any subsequent 3×3 convolution will now have $4\times$ fewer input channels, significantly reducing parameters and FLOPs.

Final Answer: 1. $14 \times 14 \times 64$. 2. Reduces channel depth ($256 \to 64$).

Question 56

Question 56: Stride Effect on Dimensions
Input: 224×224. We apply a convolution with Stride $S = 2$. Approximate the output size relative to the input. Does this increase or decrease the information density per pixel?

Solution

Concept: Stride > 1 downsamples the spatial dimensions. $W_{out} \approx W_{in}/S$.
Step-by-Step Solution:

1. **Approximation:** $224/2 = 112$. Output $\approx 112 \times 112$.

2. **Information Density:** The spatial resolution decreases (4x fewer pixels), but usually the number of channels increases in such architectures. This forces the network to compress information into fewer spatial locations (higher semantic density).

Final Answer: Output size is halved (112×112). Density increases.

Question 57

Question 57: Receptive Field Calculation
Consider two stacked 3×3 convolutional layers (Stride 1).

- Layer 1 sees 3×3 pixels of the input.
- Layer 2 sees 3×3 pixels of Layer 1's output.

What is the "Effective Receptive Field" of a single neuron in Layer 2 with respect to the original input? Hint: Trace back the inputs.

Solution

Concept: A 3×3 convolution increases the receptive field by $(K-1)$ at each step (for Stride 1). RF $= 1 + \sum(K_l - 1)$.
Step-by-Step Solution:

1. **Layer 1:** One pixel depends on a 3×3 patch. RF $= 3$.
2. **Layer 2:** One pixel depends on a 3×3 patch of Layer 1. The center of that patch sees the center of the input. The edges extend by 1 pixel on each side. Total extension: Layer 1 adds 2. Layer 2 adds 2. RF $= 1 + 2 + 2 = 5$.
3. **Visualization:** Two 3×3 layers cover a 5×5 area.

Final Answer: 5×5.

Question 58

Question 58: FLOPs Calculation
For a single convolution operation: Input $H \times W \times C_{in}$. Kernel $K \times K \times C_{in}$. Output $H \times W \times C_{out}$. The number of FLOPs is roughly $2 \times H \times W \times (K^2 \cdot C_{in}) \times C_{out}$. Calculate the FLOPs for: $H = W = 32$, $K = 3$, $C_{in} = 64$, $C_{out} = 128$.

Solution

Concept: Each output pixel requires a dot product of size $K^2 C_{in}$. There are $H \times W \times C_{out}$ output pixels.
Step-by-Step Solution:

1. **Kernel Volume:** $3 \times 3 \times 64 = 576$.
2. **Ops per Output Pixel:** 576×2 (Multiply-Add) $= 1152$.
3. **Total Output Pixels:** $32 \times 32 \times 128 = 131,072$.

4. **Total FLOPs:** $1152 \times 131,072 \approx 150,994,944$. ≈ 151 MFLOPs.

Final Answer: ≈ 151 Million FLOPs.

Question 59

Question 59: Flattening Layer
A CNN output volume is $7 \times 7 \times 512$. This volume is flattened into a 1D vector to feed into a Fully Connected (Dense) layer with 4096 neurons.

1. What is the size of the flattened vector?
2. How many weights are in the Dense layer connection?

Solution

Concept: Flattening converts 3D tensor to 1D vector. Size $= H \times W \times C$.
Step-by-Step Solution:

1. **Vector Size:** $7 \times 7 \times 512 = 49 \times 512 = 25,088$.
2. **Weights:** Input Size: 25,088. Output Size: 4,096. Weights $= 25,088 \times 4,096 \approx 102,760,448$. ($\approx 102$ Million parameters).

Final Answer: 1. 25,088. 2. ≈ 102 Million.

Question 60

Question 60: Dilated Convolution
Dilated convolution expands the kernel by inserting holes (zeros) between weights. If the dilation rate is $d = 2$ for a 3×3 kernel:

1. What is the effective size of the kernel?
2. Why is this useful for semantic segmentation?

Solution

Concept: Effective Size $K_{eff} = K + (K-1)(d-1)$.
Step-by-Step Solution:

1. **Calculation:** $3 + (2)(1) = 5$. The kernel covers a 5×5 area but still only has $3 \times 3 = 9$ parameters.
2. **Utility:** It increases the Receptive Field exponentially without losing resolution (no downsampling/pooling) and without increasing parameters. This is critical for segmentation where spatial resolution must be preserved.

Final Answer: 1. 5×5. 2. Large receptive field with no resolution loss.

7 Module 7: Recurrent Neural Networks (RNN)

Question 61

Question 61: Hidden State Calculation
Consider a simple RNN cell with a hidden state size of 2 ($h \in \mathbb{R}^2$) and input size of 2 ($x \in \mathbb{R}^2$). The update equation is:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

Given: $W_{hh} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$, $W_{xh} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $b = [0, 0]$. Initial state $h_0 = [0, 0]$. Input sequence $x_1 = [1, 0]$. Calculate the new hidden state h_1.

Solution

Concept: The hidden state carries information from the past (h_{t-1}) combined with the current input (x_t).
Step-by-Step Solution:

1. **Calculate Linear Terms:** $W_{hh}h_0 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. $W_{xh}x_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.
2. **Sum + Bias:** $z_1 = [0, 0] + [1, 0] + [0, 0] = [1, 0]$.
3. **Activation (Tanh):** $h_1 = [\tanh(1), \tanh(0)]$. $\tanh(0) = 0$. $\tanh(1) \approx 0.76$.

Final Answer: $h_1 \approx [0.76, 0]$.

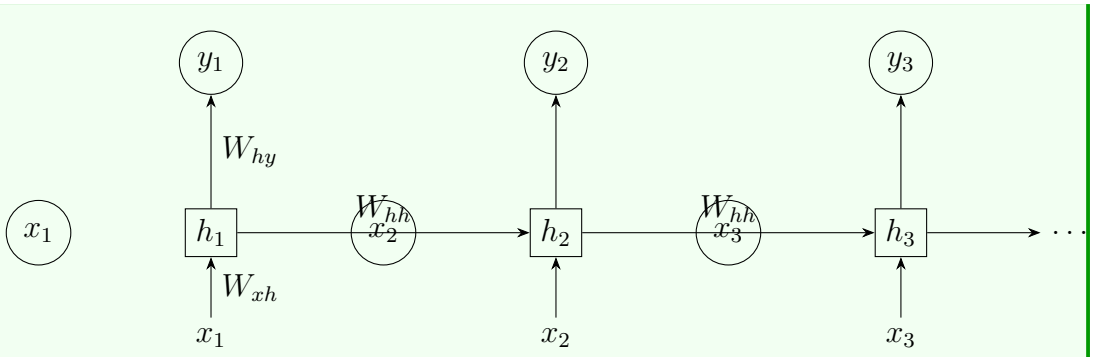
Question 62

Question 62: Sequence Unrolling
An RNN processes a sentence of length $T = 5$ ("The cat sat on mat"). Draw the unrolled computational graph. How many times are the weights W_{hh}, W_{xh}, W_{hy} reused?

Solution

Concept: RNNs share the same set of weights across all time steps. Unrolling shows the dependency over time.
Step-by-Step Solution:

1. **Graph Structure:**



2. **Weight Reuse:** The weights are reused at every time step. For $T = 5$, the weights are applied 5 times. This parameter sharing allows the model to handle sequences of variable length.

Final Answer: 5 times.

Question 63

Question 63: Parameter Counting (Simple RNN)
An RNN has:

- Input size $D = 100$.
- Hidden state size $H = 256$.
- Output size $O = 10$.

Calculate the total number of learnable parameters.

Solution

Concept: Parameters in RNN: W_{xh} (Input to Hidden): $D \times H$. W_{hh} (Hidden to Hidden): $H \times H$. b_h (Hidden Bias): H. W_{hy} (Hidden to Output): $H \times O$. b_y (Output Bias): O.
Step-by-Step Solution:

1. **Input-Hidden:** $100 \times 256 = 25,600$.
2. **Hidden-Hidden:** $256 \times 256 = 65,536$.
3. **Hidden Bias:** 256.
4. **Output Layer:** $256 \times 10 = 2,560$. Bias: 10.
5. **Total:** $25,600 + 65,536 + 256 + 2,560 + 10 = 93,962$.

Final Answer: 93,962 parameters.

Question 64

Question 64: Backpropagation Through Time (BPTT)
Why does BPTT suffer from the vanishing gradient problem for long sequences? Use the recurrence relation $h_t = \sigma(W_{hh}h_{t-1} + \dots)$.

Solution

Concept: The gradient for a weight at time t depends on the product of derivatives from time T back to t.
Step-by-Step Solution:

1. **Chain Rule:** The gradient involves a term like $\prod_{k=t}^{T} \frac{\partial h_k}{\partial h_{k-1}}$.
2. **Jacobian:** $\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(\sigma'(z_k))W_{hh}$.
3. **Magnitude:** If the largest eigenvalue of W_{hh} is < 1 (or activation derivative < 1), the product decays exponentially with $T - t$. For $T = 100$, even a value of 0.9 becomes $0.9^{100} \approx 0.00002$.

Final Answer: Gradients decay exponentially with sequence length.

Question 65

Question 65: LSTM Gate Logic
The Forget Gate in an LSTM is defined as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

If $f_t \approx 0$ for a particular memory cell, what happens to the information stored in the cell state C_{t-1}?

Solution

Concept: The cell state update is $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$.
Step-by-Step Solution:

1. **Multiplication:** Since $f_t \approx 0$, the term $f_t \odot C_{t-1}$ becomes approximately 0.
2. **Result:** The previous cell state C_{t-1} is "forgotten" (erased). The new state C_t will depend only on the new input information (from the Input Gate i_t).

Final Answer: The information is erased.

Question 66

Question 66: Gate-wise LSTM Parameter Derivation
A deep learning researcher is designing a single-layer LSTM network for a sequence processing task. The LSTM layer has an input dimension of $x = 1024$ and a hidden state dimension of $h = 512$. The model is unidirectional. Calculate the number of parameters required for each individual gate and the total number of parameters. Show the gate-wise necessary calculations for:

1. Forget Gate parameters (W_f, U_f, b_f)
2. Input Gate parameters (W_i, U_i, b_i)
3. Cell Update / Candidate parameters (W_c, U_c, b_c)
4. Output Gate parameters (W_o, U_o, b_o)
5. Total parameters for the LSTM layer.

Solution

Concept: For each gate in an LSTM, there are three types of learnable parameters:

- Input Weights (W): Connect input x_t to the gate. Dimensions: $h \times x$.
- Recurrent Weights (U): Connect previous hidden state h_{t-1} to the gate. Dimensions: $h \times h$.
- Bias (b): Dimensions: h.

Total parameters per gate $= (h \times x) + (h \times h) + h = h(x + h + 1)$.
Step-by-Step Solution:
Given: Input $x = 1024$, Hidden $h = 512$.
1. Forget Gate (f_t)

$$P_f = (512 \times 1024) + (512 \times 512) + 512$$

$$P_f = 524,288 + 262,144 + 512 = 786,944$$

2. Input Gate (i_t)

$$P_i = 786,944 \quad \text{(Same structure as Forget Gate)}$$

3. Cell Update / Candidate $(\tilde{C}_t)$

$$P_c = 786,944 \quad \text{(Same structure)}$$

4. Output Gate (o_t)

$$P_o = 786,944 \quad \text{(Same structure)}$$

5. Total Parameters Sum of all 4 gates:

$$P_{total} = 4 \times 786,944 = 3,147,776$$

Final Answer:

- Per Gate: 786,944 parameters.
- Total: 3,147,776 parameters.

Question 67

Question 67: Bidirectional RNN
A Bidirectional RNN (BiRNN) processes a sequence of length $T = 10$ with hidden size $H = 128$. The forward hidden state is $\vec{h}_t$ and backward is $\overleftarrow{h}_t$. If the final output at time t is the concatenation of both states ($y_t = [\vec{h}_t; \overleftarrow{h}_t]$), what is the dimension of y_t?

Solution

Concept: Concatenation stacks vectors.
Step-by-Step Solution:

1. **Forward State:** Size 128.
2. **Backward State:** Size 128.
3. **Concatenation:** Size $= 128 + 128 = 256$.

Final Answer: 256 dimensions.

Question 68

Question 68: Many-to-One Architecture
Give an example of a "Many-to-One" RNN architecture task. Describe the input sequence and the output.

Solution

Concept: Sequence Input $\rightarrow$ Single Output.
Step-by-Step Solution:

1. **Task:** Sentiment Analysis.
2. **Input:** A sentence (sequence of words) of variable length. "This movie was great."
3. **Output:** A single classification label (Positive/Negative).
4. **Mechanism:** The RNN processes the whole sequence, and the final hidden state h_T is used to predict the label.

Question 69

Question 69: GRU vs LSTM
Gated Recurrent Units (GRU) are a variant of LSTM.

1. How many gates does a GRU have?
2. Does GRU have a separate Cell State C_t?

Solution

Concept: Simplification of LSTM.
Step-by-Step Solution:

1. **Gates:** GRU has 2 gates: Reset Gate (r_t) and Update Gate (z_t). (LSTM has 3 gates + cell update).
2. **Cell State:** No, GRU merges the cell state and hidden state into a single state h_t.

Question 70

Question 70: Exploding Gradients Solution
If the gradients in an RNN become extremely large (NaN or Infinity) during training (Exploding Gradients), what is the standard technique to fix this? Explain "Gradient Clipping".

Solution

Concept: Limit the norm of the gradient vector.
Step-by-Step Solution:

1. **Technique:** Gradient Clipping.
2. **Algorithm:** Calculate the L2 norm of the gradient vector $\|g\|$. If $\|g\| >$ Threshold (e.g., 5):
$$g_{new} = g \times \frac{\text{Threshold}}{\|g\|}$$
3. **Effect:** This scales down the gradient vector to have a maximum length of 5, preserving its direction but preventing numerical overflow.

8 Module 8: Attention Mechanism

Question 71

Question 71: The Bottleneck Problem
In a standard Encoder-Decoder RNN (Seq2Seq) without attention, the encoder compresses the entire input sequence into a single fixed-length context vector c. Why is this a problem for translating long sentences (e.g., 50 words)?

Solution

Concept: Information bottleneck.
Step-by-Step Solution:

1. **Compression:** The fixed-size vector c must store all the semantic and syntactic details of the 50-word sentence.
2. **Loss:** As the sequence length increases, the ability of a fixed-size vector to retain information from the beginning of the sentence diminishes (vanishing gradients/forgetting).
3. **Result:** The translation quality degrades for long sentences because the decoder receives a "blurred" or incomplete representation of the input.

Question 72

Question 72: Attention Score Calculation (Dot Product)
Calculate the dot-product attention score for a Query vector $q = [1, 0, 1]$ and a Key vector $k = [0, 1, 1]$. Formula: $e = q \cdot k^T$.

Solution

Concept: Dot product measures similarity.
Step-by-Step Solution:

1. **Vectors:** $q = [1, 0, 1]$. $k = [0, 1, 1]$.
2. **Dot Product:** $1(0) + 0(1) + 1(1) = 0 + 0 + 1 = 1$.
3. **Result:** Score $e = 1$.

Question 73

Question 73: Softmax Normalization
Given raw attention scores $e = [2.0, 1.0, 0.0]$ for three keys. Calculate the attention weights α using the Softmax function. (Use $e^2 \approx 7.4, e^1 \approx 2.7, e^0 = 1$).

Solution

Concept: Weights must sum to 1.
Step-by-Step Solution:

1. **Exponentials:** $e^{2.0} \approx 7.4$. $e^{1.0} \approx 2.7$. $e^{0.0} = 1.0$.

2. **Sum:** $7.4 + 2.7 + 1.0 = 11.1$.
3. **Weights:** $\alpha_1 = 7.4/11.1 \approx 0.67$. $\alpha_2 = 2.7/11.1 \approx 0.24$. $\alpha_3 = 1.0/11.1 \approx 0.09$.
4. **Check:** $0.67 + 0.24 + 0.09 = 1.0$.

Final Answer: $\alpha = [0.67, 0.24, 0.09]$.

Question 74

Question 74: Context Vector Calculation
Using the weights from Question 73 ($\alpha = [0.67, 0.24, 0.09]$) and Values $v_1 = 10, v_2 = 20, v_3 = 0$ (assume 1D values for simplicity). Calculate the Context Vector c.

$$c = \sum \alpha_i v_i$$

Solution

Concept: Weighted sum of values.
Step-by-Step Solution:

1. **Calculation:** $c = (0.67 \times 10) + (0.24 \times 20) + (0.09 \times 0)$. $c = 6.7 + 4.8 + 0 = 11.5$.
2. **Interpretation:** The context vector is a blend of values, dominated by v_1 (highest weight).

Final Answer: $c = 11.5$.

Question 75

Question 75: Query, Key, Value Definitions
In the context of database retrieval (analogy for Attention), define:

1. Query (Q).
2. Key (K).
3. Value (V).

Solution

Concept: Retrieval analogy.
Step-by-Step Solution:

1. **Query (Q):** What you are looking for (e.g., the current decoder state trying to find relevant info).
2. **Key (K):** The label or identifier of the information in the database (e.g., encoder hidden states). Used to match against the Query.
3. **Value (V):** The actual content associated with the Key (e.g., the same encoder hidden states). Used to construct the output.

Question 76

Question 76: Scaled Dot-Product Attention
The formula for Scaled Dot-Product Attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Why do we divide by $\sqrt{d_k}$? Assume $d_k = 100$.

Solution

Concept: Gradient stability.
Step-by-Step Solution:

1. **Magnitude:** If d_k is large, the dot product $Q \cdot K^T$ can result in very large values.
2. **Softmax Saturation:** Large inputs to Softmax push the function into regions where gradients are extremely small (close to 0 or 1).
3. **Scaling:** Dividing by $\sqrt{100} = 10$ reduces the scale of the dot products, keeping them in a range where Softmax has useful gradients (linear regime).

Question 77

Question 77: Bahdanau (Additive) Attention
Bahdanau Attention computes scores using a feed-forward network:

$$e_{ij} = v_a^T \tanh(W_a[s_{i-1}; h_j])$$

Count the number of weight matrices involved in this computation compared to Dot-Product Attention.

Solution

Concept: Additive vs Multiplicative.
Step-by-Step Solution:

1. **Bahdanau:** Uses W_a (to combine states) and v_a (to project to scalar). Generally implemented as $W_s s_{i-1} + W_h h_j$. Requires learnable parameters (W_s, W_h, v_a).
2. **Dot-Product:** Uses no learnable parameters for the score calculation itself (just matrix multiplication).
3. **Comparison:** Bahdanau is more computationally expensive and has more parameters.

Question 78

Question 78: Self-Attention
In Self-Attention, where do the Queries, Keys, and Values come from?

Solution

Concept: Internal sequence modeling.
Step-by-Step Solution:

1. **Source:** They all come from the *same* input sequence (hence "Self").
2. **Transformation:** The input vector x_i is multiplied by three different weight matrices (W^Q, W^K, W^V) to produce q_i, k_i, v_i.
3. **Purpose:** To relate different positions of the same sequence to compute a representation of the sequence.

Question 79

Question 79: Hard vs Soft Attention
Explain the difference between Hard Attention and Soft Attention. Which one is differentiable?

Solution

Concept: Stochastic vs Deterministic.
Step-by-Step Solution:

1. **Soft Attention:** Calculates a weighted average (expected value) using Softmax probabilities. It is fully differentiable and can be trained with standard Backpropagation.
2. **Hard Attention:** Stochastically samples *one* region to attend to (weight is 1 for chosen, 0 for others). It is not differentiable (requires Reinforcement Learning / REINFORCE to train).

Question 80

Question 80: Global vs Local Attention
What is the computational complexity of Global Attention for a sequence of length N? Why might Local Attention be preferred for very long sequences?

Solution

Concept: $O(N^2)$ vs $O(N \cdot Window)$.
Step-by-Step Solution:

1. **Global Complexity:** Every token attends to every other token. Complexity is $O(N^2)$.
2. **Local Preference:** For very long sequences (e.g., documents), N^2 is prohibitive (memory and time). Local attention restricts the window to a small

size D. Complexity becomes $O(N \cdot D)$, which is linear in N.

9 Module 9: Transformer

Question 81

Question 81: Self-Attention Matrix Dimensions
In a Transformer block, the input sequence length is $N = 10$ and the embedding dimension is $d_{model} = 64$. We compute the Query (Q), Key (K), and Value (V) matrices.

1. What are the dimensions of Q, K, V?
2. What is the dimension of the raw attention score matrix $S = QK^T$?
3. What is the dimension of the final output matrix $Z = \text{softmax}(S)V$?

Solution

Concept: Matrix multiplication rules.
Step-by-Step Solution:

1. **Q, K, V Dimensions:** Each is a projection of the input X ($N \times d_{model}$). Usually $d_k = d_v = d_{model}$ (or smaller in multi-head). Assume single head: $Q, K, V \in \mathbb{R}^{10\times 64}$.
2. **Score Matrix S:** Q is 10×64. K^T is 64×10. $S = QK^T$ results in 10×10. (This is the $N \times N$ attention map).
3. **Output Matrix Z:** $\text{softmax}(S)$ is 10×10. V is 10×64. $Z = S \times V$ results in 10×64. (Same size as input X).

Final Answer: 1. 10×64. 2. 10×10. 3. 10×64.

Question 82

Question 82: Self-Attention Output Calculation
Consider a simplified Self-Attention mechanism with:

- Input sequence of 2 tokens: $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (Rows are tokens, $d_{model} = 2$).
- Weight matrices (Identity for simplicity): $W^Q = W^K = W^V = I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$.

Calculate the final output Z of the self-attention layer. Steps:

1. Compute Q, K, V.
2. Compute raw attention scores $S = QK^T$.
3. Compute scaled scores (assume scaling factor $\sqrt{d_k} = 1$ for simplicity).
4. Compute Softmax weights (Use $e^1 \approx 2.7, e^0 = 1$).
5. Compute final output $Z = \text{softmax}(S)V$.

Solution

Concept: $Z = \text{softmax}(QK^T)V$.
Step-by-Step Solution:

1. Compute Q, K, V Since $W = I$, $Q = X, K = X, V = X$. $Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$.
2. Compute Scores $S = QK^T$

$$S = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

3. Compute Softmax Row 1 ($[1, 0]$): $e^1 \approx 2.7, e^0 = 1$. Sum $= 3.7$. Weights: $[2.7/3.7, 1/3.7] \approx [0.73, 0.27]$.
Row 2 ($[0, 1]$): $e^0 = 1, e^1 \approx 2.7$. Sum $= 3.7$. Weights: $[1/3.7, 2.7/3.7] \approx [0.27, 0.73]$.
Attention Matrix $A \approx \begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$.
4. Compute Output $Z = AV$

$$Z = \begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$$

Final Answer: $Z \approx \begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$.

Question 83

Question 83: Positional Encoding Calculation
The Positional Encoding formula is:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Calculate the value for Position $pos = 1$ and Dimension index $2i = 0$ (i.e., the first dimension) with $d_{model} = 512$.

Solution

Concept: Unique encoding for each position.
Step-by-Step Solution:

1. **Substitute values:** $pos = 1$. $2i = 0 \Rightarrow i = 0$.
2. **Calculate Argument:** Denominator: $10000^{0/512} = 10000^0 = 1$. Argument: $1/1 = 1$.
3. **Calculate Sine:** $\sin(1) \approx 0.8415$ (radians).

Final Answer: $PE_{(1,0)} \approx 0.84$.

Question 84

Question 84: Masked Self-Attention
In the Decoder of a Transformer (like GPT), we use "Masked" Self-Attention. What does the mask look like for a sequence of length 4? Explain why masking is necessary during training.

Solution

Concept: Causal masking (prevent peeking into the future).
Step-by-Step Solution:

1. **Mask Structure:** Upper triangular matrix (excluding diagonal) is set to $-\infty$.
$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$
2. **Necessity:** During training, we feed the entire target sequence. Without masking, position t could attend to position $t+1$ (the answer), making the task trivial and preventing the model from learning to predict the next token based *only* on past tokens.

Question 85

Question 85: Feed-Forward Network in Transformer
The FFN in a Transformer block consists of two linear layers with a ReLU activation in between:
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$
If input dimension is $d_{model} = 512$ and inner dimension is $d_{ff} = 2048$: Calculate the number of parameters in this sub-layer.

Solution

Concept: Parameter counting for dense layers.
Step-by-Step Solution:

1. **Layer 1 (W_1):** $512 \times 2048 + 2048 = 1,048,576 + 2,048 = 1,050,624$.
2. **Layer 2 (W_2):** $2048 \times 512 + 512 = 1,048,576 + 512 = 1,049,088$.
3. **Total:** $1,050,624 + 1,049,088 = 2,099,712$. ($\approx 2.1$ Million).

Final Answer: ≈ 2.1 Million.

Question 86

Question 86: Complexity Analysis (RNN vs Transformer)
Compare the computational complexity per layer for a sequence of length N and dimension d.

- RNN: $O(N \cdot d^2)$.
- Self-Attention: $O(N^2 \cdot d)$.

If $N = 1000$ and $d = 512$, which is computationally heavier?

Solution

Concept: Complexity trade-off.
Step-by-Step Solution:

1. **RNN:** $1000 \times 512^2 = 1000 \times 262,144 \approx 2.6 \times 10^8$.
2. **Self-Attention:** $1000^2 \times 512 = 1,000,000 \times 512 \approx 5.1 \times 10^8$.
3. **Conclusion:** Self-Attention is heavier for long sequences when $N > d$. Here, $N(1000) > d(512)$, so Attention is more expensive.

Final Answer: Self-Attention is heavier.

Question 87

Question 87: Layer Normalization
Transformers use Layer Normalization instead of Batch Normalization. Given a hidden vector $h = [1, 2, 3]$ for a single token. Calculate the Layer Norm output (assume $\gamma = 1, \beta = 0, \epsilon = 0$).

$$\hat{h} = \frac{h - \mu}{\sigma}$$

Solution

Concept: Normalization across the feature dimension.
Step-by-Step Solution:

1. **Mean μ:** $(1 + 2 + 3)/3 = 2$.
2. **Variance σ^2:** $((1-2)^2 + (2-2)^2 + (3-2)^2)/3 = (1+0+1)/3 = 2/3 \approx 0.67$. Standard Deviation $\sigma = \sqrt{0.67} \approx 0.816$.
3. **Normalize:** $h_1 : (1-2)/0.816 = -1.22$. $h_2 : (2-2)/0.816 = 0$. $h_3 : (3-2)/0.816 = 1.22$.

Final Answer: $[-1.22, 0, 1.22]$.

Question 88

Question 88: Encoder-Decoder Architecture
In the original Transformer (Vaswani et al.), the decoder has a third sub-layer that the encoder does not have. What is it and what is its function?

Solution

Concept: Cross-Attention.
Step-by-Step Solution:

1. **Sub-layer:** Encoder-Decoder Attention (Cross-Attention).
2. **Function:** It allows the decoder to attend to the output of the *encoder*. The Queries (Q) come from the previous decoder layer. The Keys (K) and Values (V) come from the encoder output. This aligns the generation step with the relevant parts of the input source.

Question 89

Question 89: BERT vs GPT
BERT and GPT are both Transformer-based models.

1. Which one uses the Encoder stack?
2. Which one uses the Decoder stack?
3. Which one is bidirectional?

Solution

Concept: Architectural differences.
Step-by-Step Solution:

1. **BERT:** Uses **Encoder** stack. It is **Bidirectional** (Masked Language Model).
2. **GPT:** Uses **Decoder** stack. It is **Unidirectional** (Auto-regressive, Left-to-Right).

Question 90

Question 90: Residual Connections
Why are Residual (Skip) Connections crucial for training deep Transformers? Formula: $x_{out} = \text{LayerNorm}(x + \text{SubLayer}(x))$.

Solution

Concept: Gradient flow highway.
Step-by-Step Solution:

1. **Problem:** In very deep networks, gradients vanish as they propagate through many non-linear layers.
2. **Solution:** The skip connection $x + F(x)$ allows gradients to flow directly through the identity path (x) without being attenuated by the non-linear function $F(x)$.
3. **Result:** Enables training of hundreds of layers (e.g., GPT-3 has 96 layers).

10 Module 10: Optimizers

Question 91

Question 91: Momentum Update
Calculate the updated velocity v_t and weight w_t using Momentum SGD. Given:

- Learning Rate $\eta = 0.1$.
- Momentum coefficient $\gamma = 0.9$.
- Current velocity $v_{t-1} = 2$.
- Current gradient $g_t = 1$.

Formula:

$$v_t = \gamma v_{t-1} + \eta g_t$$

$$w_t = w_{t-1} - v_t$$

Solution

Concept: Momentum accumulates past gradients to accelerate in relevant directions and dampen oscillations.
Step-by-Step Solution:

1. **Calculate Velocity:** $v_t = (0.9 \times 2) + (0.1 \times 1)$. $v_t = 1.8 + 0.1 = 1.9$.
2. **Calculate Weight Update:** Let's say $w_{t-1} = 0$. $w_t = 0 - 1.9 = -1.9$.
3. **Comparison:** Without momentum, update would be $0.1 \times 1 = 0.1$. With momentum, update is 1.9 (much larger, continuing previous motion).

Final Answer: $v_t = 1.9$.

Question 92

Question 92: Saddle Points
In high-dimensional non-convex optimization (like deep learning), are Local Minima or Saddle Points a bigger problem? Explain why gradients vanish at a saddle point.

Solution

Concept: Curvature analysis (Hessian).
Step-by-Step Solution:

1. **Problem:** Saddle points are much more common than local minima in high dimensions.
2. **Gradient:** At a saddle point, the gradient is zero (flat), just like at a minimum. However, in some directions, the function curves up (min), and in others, it curves down (max).
3. **Escape:** SGD can get stuck temporarily, but noise usually helps escape.

Second-order methods can detect saddle points by checking eigenvalues of the Hessian (mixed signs).

Final Answer: Saddle points are more common. Gradients vanish because it is a critical point.

Question 93

Question 93: Adagrad Weakness
Adagrad adapts the learning rate for each parameter by dividing by the square root of the sum of squared past gradients:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

where $G_t = \sum_{\tau=1}^{t} g_\tau^2$. What happens to the effective learning rate as training progresses (large t)?

Solution

Concept: Monotonically increasing denominator.
Step-by-Step Solution:

1. **Accumulation:** Since squared gradients are always positive, G_t increases monotonically with every step.
2. **Decay:** As G_t grows very large, the denominator $\sqrt{G_t}$ becomes huge.
3. **Result:** The effective learning rate $\frac{\eta}{\sqrt{G_t}}$ shrinks to zero, causing the algorithm to stop learning prematurely, even if it hasn't reached the minimum.

Final Answer: It decays to zero excessively fast.

Question 94

Question 94: RMSProp Solution
How does RMSProp fix the aggressive decay of Adagrad? Formula: $E[g^2]_t = \rho E[g^2]_{t-1} + (1-\rho)g_t^2$. Calculate $E[g^2]_t$ if $\rho = 0.9$, previous average $= 10$, current gradient $= 0$.

Solution

Concept: Exponential Moving Average (Leaky integration).
Step-by-Step Solution:

1. **Mechanism:** Instead of summing all past squared gradients, RMSProp uses an exponentially decaying average. This forgets old history and prevents the denominator from exploding.
2. **Calculation:** $E[g^2]_t = (0.9 \times 10) + (0.1 \times 0^2)$. $E[g^2]_t = 9 + 0 = 9$.
3. **Result:** The accumulated squared gradient decreases (from 10 to 9) because

the current gradient is small, allowing the learning rate to increase again if needed.

Final Answer: $E[g^2]_t = 9$.

Question 95

Question 95: Adam Optimizer Calculation
Adam combines Momentum and RMSProp. Compute the bias-corrected first moment estimate $\hat{m}_t$. Given:

- Current Step $t = 1$.
- Decay rate $\beta_1 = 0.9$.
- Current Gradient $g_t = 10$.
- Initial Moment $m_0 = 0$.

Formula:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

Solution

Concept: Bias correction counteracts initialization at zero.
Step-by-Step Solution:

1. **Calculate Raw Moment m_1:** $m_1 = (0.9 \times 0) + (0.1 \times 10) = 1$.
2. **Calculate Bias Correction Factor:** $1 - \beta_1^1 = 1 - 0.9 = 0.1$.
3. **Calculate Corrected Moment $\hat{m}_1$:** $\hat{m}_1 = 1/0.1 = 10$.
4. **Observation:** Without correction, the estimate (1) would be much smaller than the true gradient (10) at the start. Correction scales it back up.

Final Answer: $\hat{m}_1 = 10$.

Question 96

Question 96: Learning Rate Decay
A Step Decay schedule drops the learning rate by a factor of 0.5 every 10 epochs. If initial LR is 0.1, what is the LR at epoch 25?

Solution

Concept: Discrete schedule.
Step-by-Step Solution:

1. **Initial:** $\eta_0 = 0.1$.

2. **Epoch 10:** Drop 1. $\eta = 0.1 \times 0.5 = 0.05$.
3. **Epoch 20:** Drop 2. $\eta = 0.05 \times 0.5 = 0.025$.
4. **Epoch 25:** No drop (next is 30).

Final Answer: $\eta = 0.025$.

Question 97

Question 97: Batch Size and Generalization
Why does training with a very large batch size (e.g., 8192) often lead to worse generalization (test accuracy) compared to smaller batch sizes (e.g., 32), even if training loss is similar?

Solution

Concept: Sharp vs Flat Minima.
Step-by-Step Solution:

1. **Sharp Minima:** Large batch gradients are very accurate, converging directly to the nearest minimum. In high dimensions, this is often a "sharp" minimum (narrow valley).
2. **Flat Minima:** Small batch gradients are noisy. This noise prevents the optimizer from settling in sharp valleys, pushing it towards "flat" (wide) minima.
3. **Generalization:** Flat minima are robust to shifts between train and test data distributions. Sharp minima are not. Thus, small batches generalize better.

Question 98

Question 98: Second-Order Methods (Newton's Method)
Why are second-order optimization methods (using the Hessian matrix H) rarely used in Deep Learning? Complexity analysis of inverting H.

Solution

Concept: Computational intractability.
Step-by-Step Solution:

1. **Hessian Size:** For a model with N parameters, the Hessian is an $N \times N$ matrix.
2. **Example:** If $N = 1,000,000$ (small by modern standards), $N^2 = 10^{12}$ elements. Storing this requires terabytes of memory.
3. **Inversion:** Newton's method requires computing H^{-1}. Matrix inversion is $O(N^3)$.
4. **Conclusion:** It is computationally impossible for large networks.

Question 99

Question 99: Sparse Gradients
Which optimizer is better suited for data with sparse features (e.g., NLP with rare words): SGD or Adagrad? Explain why.

Solution

Concept: Adaptive learning rates for infrequent updates.
Step-by-Step Solution:

1. **Scenario:** Rare features have gradients that are mostly zero.
2. **SGD:** Updates all parameters with the same learning rate. Rare features get very few updates, so they might not learn enough.
3. **Adagrad:** Increases the effective learning rate for parameters with small past gradients (rare features) and decreases it for frequent ones.
4. **Conclusion:** Adagrad is better for sparse data.

Question 100

Question 100: Warm-up Steps
In Transformer training, the learning rate is often increased linearly for the first few thousand steps ("Warm-up") before decaying. Why is this necessary?

Solution

Concept: Stabilizing early training variance.
Step-by-Step Solution:

1. **Initialization:** At the start, parameters are random.
2. **Gradients:** The gradients are very large and unstable.
3. **Risk:** A large learning rate immediately can cause divergence or push weights into bad regions.
4. **Warm-up:** Starting with a small LR allows the optimizer to gather statistics (like Adam's variance estimates) and move safely before accelerating.

11 Module 11: Regularisation

Question 101

Question 101: L2 Regularization Penalty Calculation
Given a weight vector $w = [3, 4]$ and a regularization strength $\lambda = 0.01$. Calculate the L2 penalty term added to the loss function. Formula: $\Omega(w) = \frac{1}{2}\lambda\|w\|_2^2$.

Solution

Concept: L2 norm squared.
Step-by-Step Solution:

1. **Norm Squared:** $\|w\|_2^2 = 3^2 + 4^2 = 9 + 16 = 25$.
2. **Penalty Term:** $\Omega(w) = 0.5 \times 0.01 \times 25$. $\Omega(w) = 0.5 \times 0.25 = 0.125$.
3. **Result:** The loss increases by 0.125.

Final Answer: 0.125.

Question 102

Question 102: L2 Weight Decay Update
Using the same $w = [3, 4]$ and $\lambda = 0.01$, calculate the updated weight w_{new} after one step of Gradient Descent if the data loss gradient is zero ($\nabla L_{data} = 0$) and learning rate $\eta = 0.1$. Formula: $w_{new} = w_{old} - \eta(\nabla L_{data} + \lambda w_{old})$.

Solution

Concept: Weight decay shrinks weights towards zero.
Step-by-Step Solution:

1. **Regularization Gradient:** $\nabla\Omega(w) = \lambda w = 0.01 \times [3, 4] = [0.03, 0.04]$.
2. **Total Gradient:** $\nabla L = 0 + [0.03, 0.04] = [0.03, 0.04]$.
3. **Update:** $w_{new} = [3, 4] - 0.1 \times [0.03, 0.04]$. $w_{new} = [3 - 0.003, 4 - 0.004]$. $w_{new} = [2.997, 3.996]$.
4. **Observation:** Both weights decreased slightly.

Final Answer: $[2.997, 3.996]$.

Question 103

Question 103: L1 Regularization and Sparsity
Explain why L1 regularization ($\lambda \sum |w_i|$) tends to produce sparse weight vectors (many weights exactly zero) compared to L2. Use the geometric intuition of the constraint region.

Solution

Concept: Diamond vs Circle constraint.
Step-by-Step Solution:

1. **Geometry:** L1 constraint shape is a diamond (vertices on axes). L2 is a circle.
2. **Optimization:** The optimal solution occurs where the loss contours touch the constraint boundary.
3. **Intersection:** The "pointy" corners of the L1 diamond are on the axes (where some weights are 0). It is highly probable for the loss contour to touch a corner first.
4. **L2:** The circle has no corners, so the touch point is almost never on an axis (weights are small but non-zero).

Question 104

Question 104: Dropout during Training
In a layer with 10 neurons, we apply Dropout with probability $p = 0.2$ (keep probability 0.8). During a training step, how many neurons are active on average? What happens to their output values to maintain magnitude?

Solution

Concept: Expected value maintenance (Inverted Dropout).
Step-by-Step Solution:

1. **Active Neurons:** $10 \times (1 - p) = 10 \times 0.8 = 8$ neurons.
2. **Scaling:** If we drop 20% of neurons, the total sum of activations will drop by 20%. To fix this, we scale the remaining activations by $\frac{1}{1-p} = \frac{1}{0.8} = 1.25$. This ensures the expected sum remains the same.

Final Answer: 8 neurons. Scaled by 1.25.

Question 105

Question 105: Dropout during Inference
During inference (testing), do we apply Dropout? If we used standard Dropout (not inverted) during training, how must we scale the weights during inference?

Solution

Concept: Deterministic inference.
Step-by-Step Solution:

1. **Apply Dropout?** No. We want deterministic outputs and to use the full capacity of the model.
2. **Scaling (Standard Dropout):** If training activations were *not* scaled up,

the expected training output was $(1-p) \times$ Actual. During inference, we have Actual (since no dropout). To match training magnitude, we must multiply weights by $(1-p)$. (This is why Inverted Dropout is preferred—it handles scaling during training).

Final Answer: No. Multiply by $(1-p)$.

Question 106

Question 106: Data Augmentation Techniques
List 3 distinct data augmentation techniques for image classification. Explain how one of them helps the model generalize to new viewpoints.

Solution

Concept: Invariance learning.
Step-by-Step Solution:

1. **Techniques:**
   - Random Rotation.
   - Random Horizontal Flip.
   - Random Crop / Zoom.
2. **Generalization (Rotation):** By training on rotated versions of an object (e.g., a car at 15°), the model learns features that are rotation-invariant. When it sees a real car at a slight angle in the test set, it can still recognize it.

Question 107

Question 107: Early Stopping
You are monitoring the Training Loss and Validation Loss curves.

- Training Loss is decreasing steadily.
- Validation Loss decreases until Epoch 50, then starts increasing.

What is happening after Epoch 50? What should you do?

Solution

Concept: Overfitting onset.
Step-by-Step Solution:

1. **Diagnosis:** The model has started to memorize the training data noise rather than learning general patterns. This is **Overfitting**.
2. **Action:** Stop training immediately (or restore the checkpoint from Epoch 50). This is **Early Stopping**.

Question 108

Question 108: Batch Normalization Benefits
Batch Normalization (BN) reduces "Internal Covariate Shift".

1. What is Internal Covariate Shift?
2. How does BN allow for higher learning rates?

Solution

Concept: Stabilizing layer distributions.
Step-by-Step Solution:

1. **Shift:** As parameters of earlier layers change, the distribution of inputs to later layers changes constantly. The later layers must keep adapting to this moving target.
2. **Higher LR:** BN constraints activations to a standard range (mean 0, var 1). This prevents gradients from exploding or vanishing, allowing us to take larger steps without diverging.

Question 109

Question 109: Ensembling (Bagging)
You train 5 different models on random subsets of the training data. During testing, you average their predictions. Why does this reduce the variance of the final prediction? Formula: $\text{Var}(\frac{1}{n}\sum X_i) = \frac{1}{n}\text{Var}(X)$.

Solution

Concept: Averaging independent errors.
Step-by-Step Solution:

1. **Assumption:** The models make *independent* errors (uncorrelated).
2. **Variance Reduction:** The variance of the mean of n independent variables is $1/n$ times the variance of a single variable.
3. **Result:** By averaging 5 models, the variance (error due to noise/randomness) is reduced by a factor of 5, leading to more stable and accurate predictions.

Question 110

Question 110: Noise Injection
Adding Gaussian noise to the *inputs* during training is equivalent to what form of regularization? Prove or Explain intuitively.

Solution

Concept: Smoothness enforcement.
Step-by-Step Solution:

1. **Equivalence:** It is equivalent to **L2 Regularization** (Tikhonov Regularization).
2. **Intuition:** To be robust to noise $x + \epsilon$, the function $f(x)$ must be flat (smooth). Ideally, $f(x + \epsilon) \approx f(x)$. By Taylor expansion, minimizing the error with noise adds a term proportional to the square of the derivative (slope). Penalizing the slope (derivative) is similar to penalizing large weights (L2).